08 图

- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

08 图

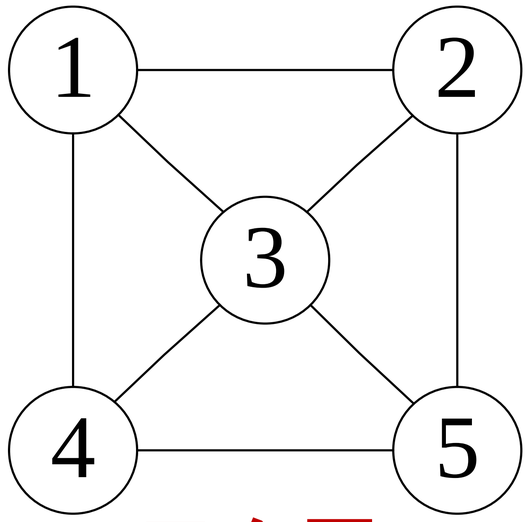
- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

图的基本概念

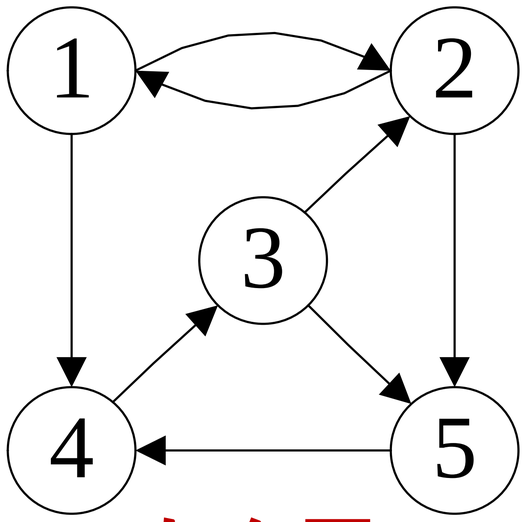
- 图(graph)G是由顶点(vertex)集合V及顶点间的关系(也叫做边(edge))集合E组成的一种数据结构，即G = (V, E)
  - V = {x | x ∈ 某个数据对象}是顶点的有穷非空集合
  - E = {(x, y) | x, y ∈ V}是顶点间关系的有穷集合

- 无向图(undirected graph)：所有的边(x, y)是无序的
  - 每对顶点之间都有一条边的无向图，叫完全无向图
  - 有n个顶点的完全无向图有n(n-1)/2条边 C_n^2
- 有向图(directed graph)：所有的边<x, y>是有序的
  - 每对顶点i和j之间都有边<i, j>和边<j, i>的有向图，叫完全有向图
  - 有n个顶点的完全有向图有n(n-1)条边 A_n^2

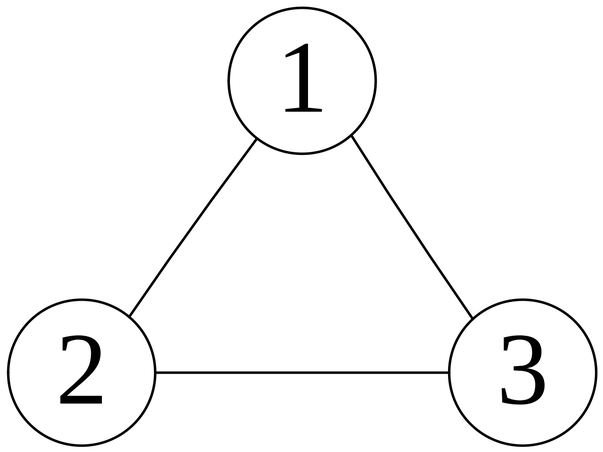
无向图与有向图示例



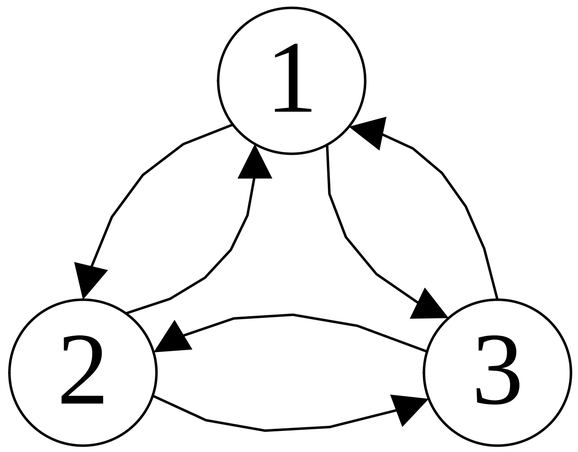
无向图



有向图



完全无向图



完全有向图

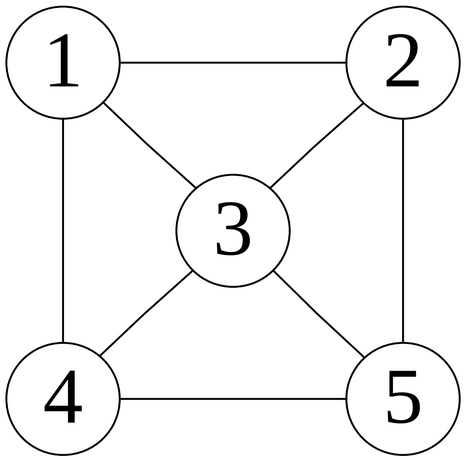
图的基本概念

- 权(weight)：边具有与它相关的数值，该数值称为权，这种带权图叫做网络(network)
  - 权可以表示从一个顶点到另一个顶点的距离、花费的代价、所需的时间、次数等

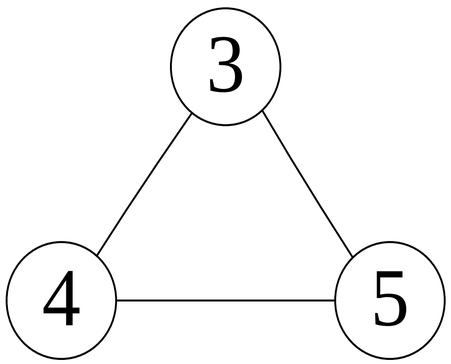
- 邻接顶点(adjacent vertex)：如果(u, v)是E(G)中的一条边，则称u与v互为邻接顶点

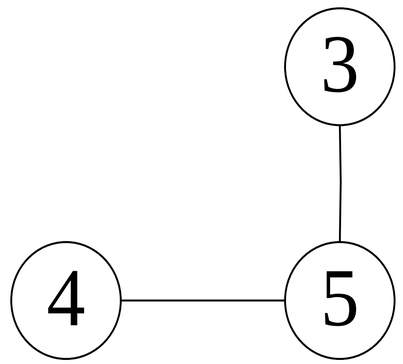
- 子图(subgraph)：对于两个图G＝(V, E)和G’＝(V’, E’), 若V’⊆V且E’⊆E，则称图G’是图G的子图

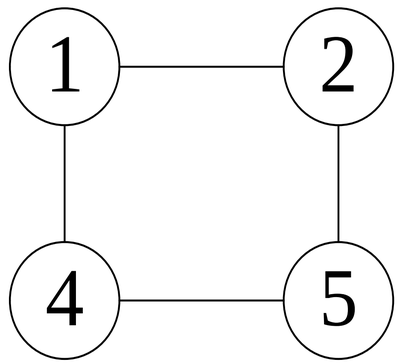
子图示例



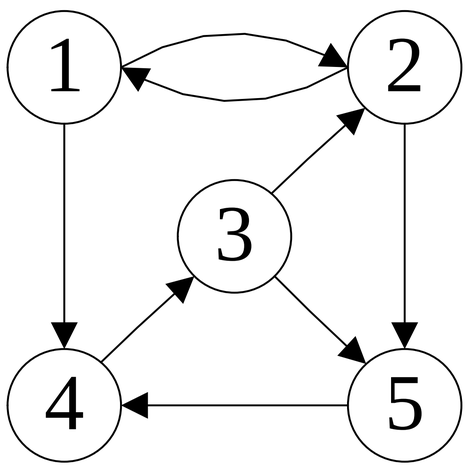
无向图G_1



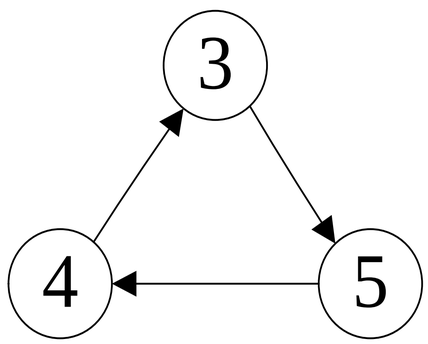


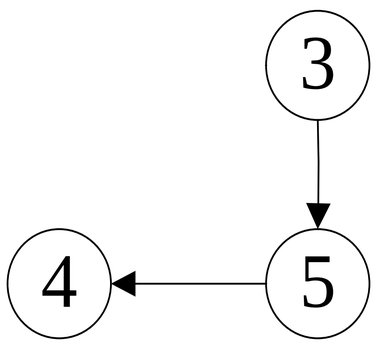


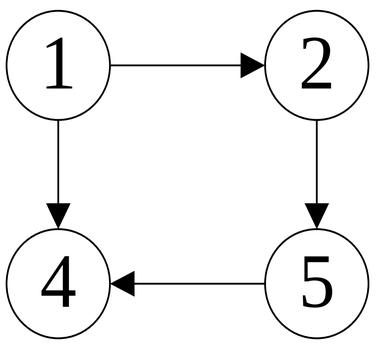
无向图G_1的某些子图



有向图G_2



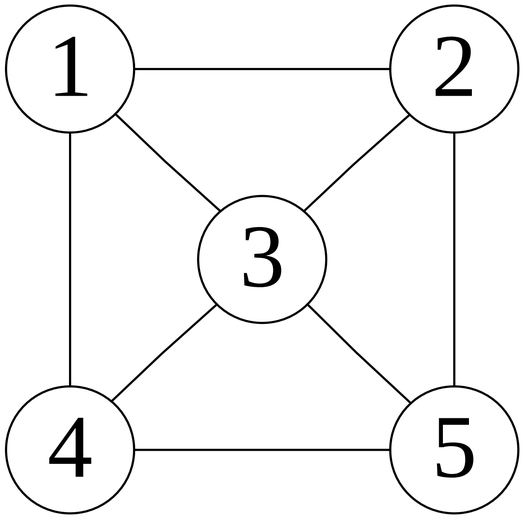


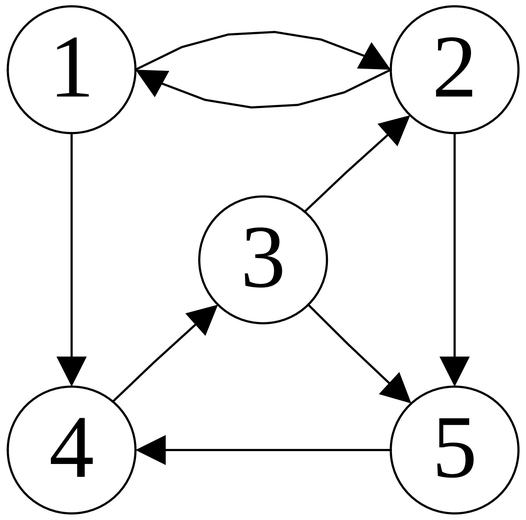


有向图G_2的某些子图

图的基本概念

- 顶点的**度**(degree)：与顶点v关联的边的条数，记作TD(v)
  - 有向图中，顶点的度等于该顶点的入度与出度之和
  - 顶点的**入度**：以顶点v为终点的有向边的条数，记作ID(v)
  - 顶点的**出度**：以顶点v为始点的有向边的条数，记作OD(v)

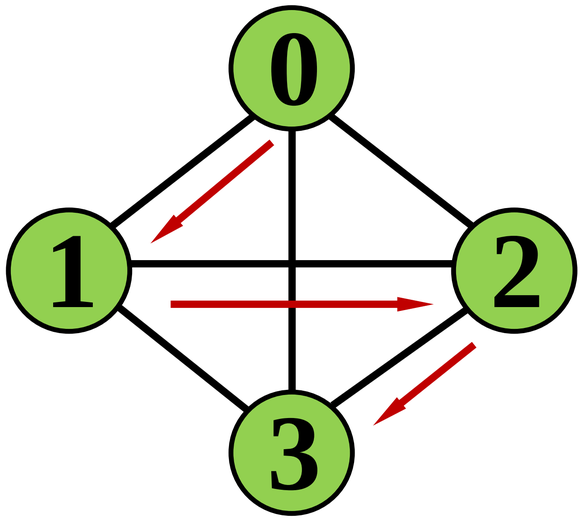




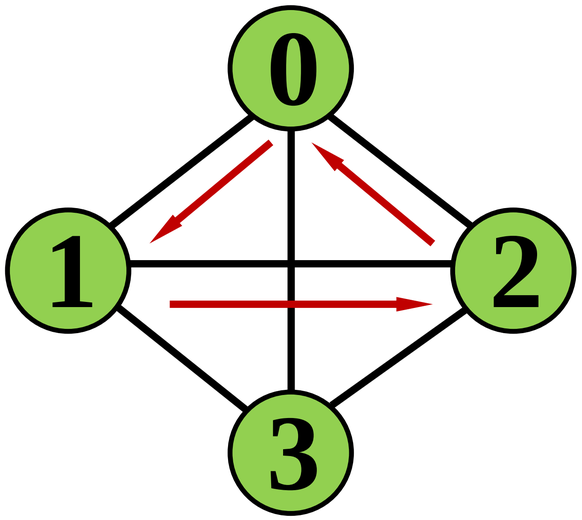
图的基本概念

- **路径(path)：在图G＝(V, E)中，从顶点v到顶点w的一条路径是由顶点组成的序列(v_0, v_1, …, v_k)，v_0＝v，v_k＝w，<v_i, v_{i+1}> ∈ E(G) (0 ≤ i ≤ k)**
- **路径长度(path length)**
  - 非带权图的路径长度是指此路径上边的条数
  - 带权图的路径长度是指路径上各边的权之和
- **简单路径：若路径上各顶点v_1, v_2, ..., v_m均不互相重复，则称这样的路径为简单路径**
- **回路(cycle)：若路径上第一个顶点v_1与最后一个顶点v_m重合，则称这样的路径为回路或环**

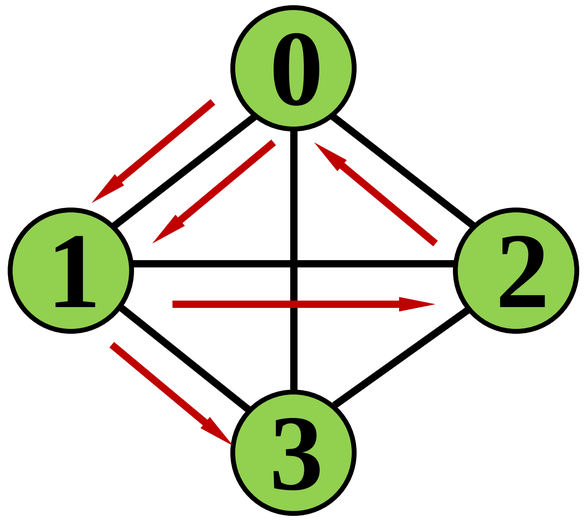
简单路径与回路示例



(0, 1, 2, 3)是简单路径，长度为3



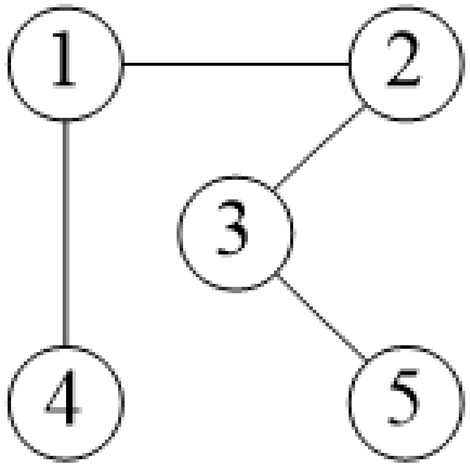
(0, 1, 2, 0)是回路，长度为3



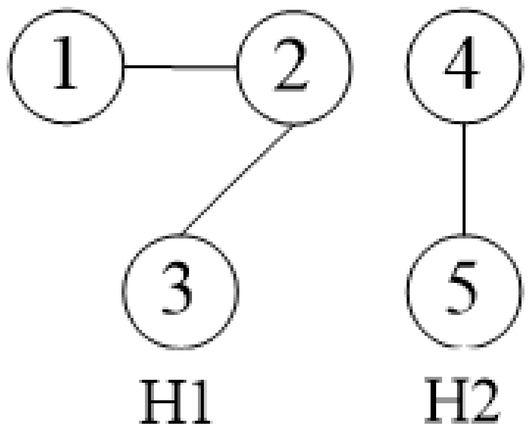
(0, 1, 2, 0, 1, 3)是非简单路径，长度为5

图的基本概念

- 若从顶点v_1到顶点v_2有路径，则称顶点v_1与v_2是连通的
- 若无向图中任意一对顶点都是连通的，则称它是连通图
- 非连通图的极大连通子图叫做连通分量



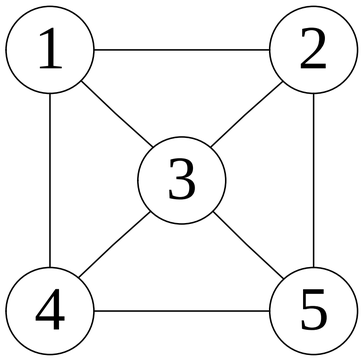
连通图



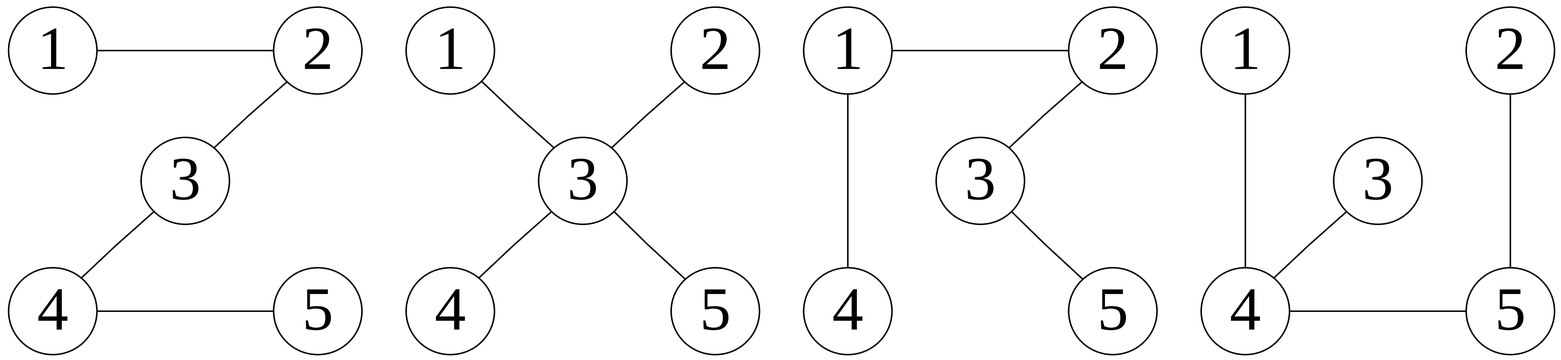
具有2个连通分量的非连通图

图的基本概念

- **一个连通图的生成树(spanning tree)是其极小连通子图，在n个顶点的情形下，有n-1条边**



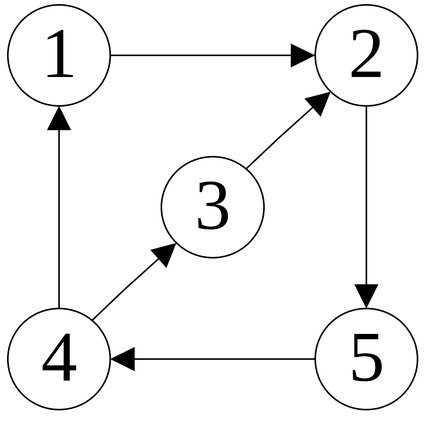
连通图G



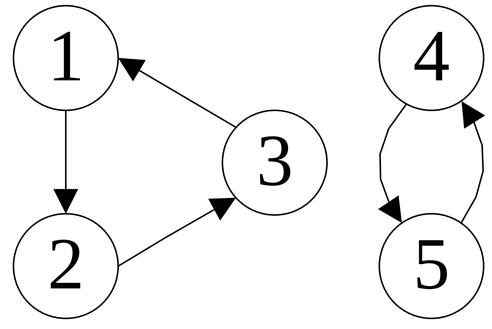
连通图G的生成树

图的基本概念

- 若**有向图**中任意一对顶点v_i和v_j，都存在一条从v_i到v_j和从v_j到v_i的路径，则称它是**强连通图**(双向连通图)
- 非强连通图的极大强连通子图叫做**强连通分量**



强连通图

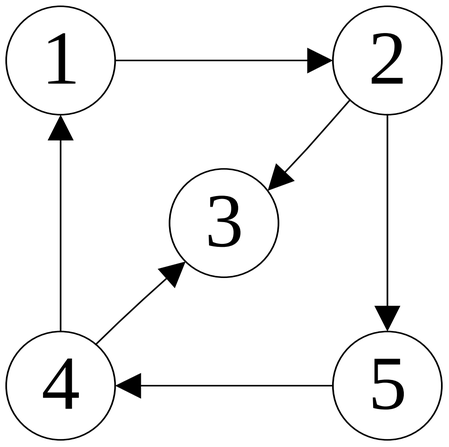


具有2个强连通分量的非强连通图

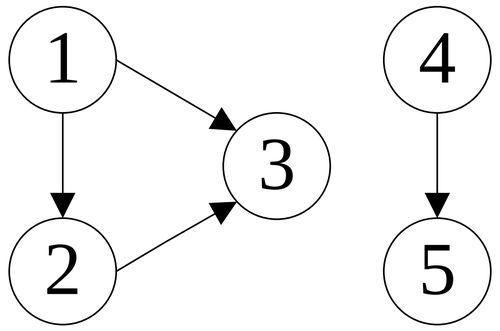
图的基本概念

把它当成无向的

- 若**有向图**中任意一对顶点v和w，有一个由不同顶点组成的序列($v_0, v_1, \ldots, v_k$)，$v_0=v$，$v_k=w$，且$<v_i, v_{i+1}>\in E(G)$或者$<v_{i+1}, v_i>\in E(G)$ $(0 \le i < k)$，则称它为**弱连通图**
- 非弱连通图的极大弱连通子图叫做**弱连通分量**



弱连通图



具有2个弱连通分量的非弱连通图

练习

- 如果有n个顶点的图形成一个环，则它有多少棵生成树？
- n棵生成树

- G是一个非连通无向图，共有28条边，则该图至少有多少个顶点？
- 图G是非连通无向图，至少有两个连通分量。一个连通分量最少的顶点数是由28条边组成的无向完全图，其顶点数n可由$e \le n(n-1)/2$，解之得$n \ge 8$；另外一个顶点自成一个连通分量，所以该图至少有9个顶点

08 图

- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

图的邻接矩阵(Adjacency Matrix)表示

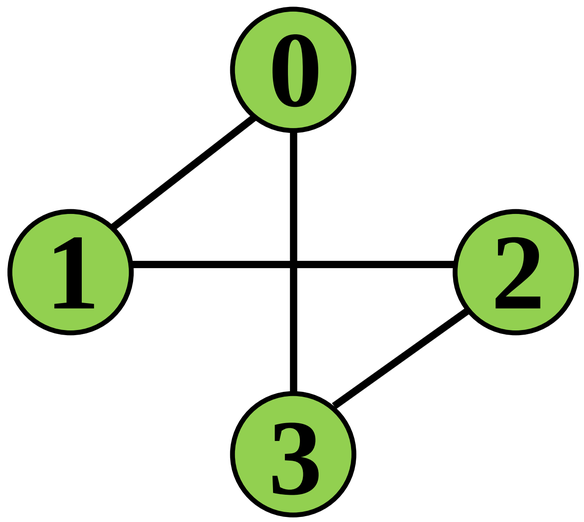
- 在图的邻接矩阵表示中，有一个记录各个顶点信息的顶点表，还有一个记录各个顶点之间关系的邻接矩阵
- 设图A=(V, E)是一个有n个顶点的图，图的邻接矩阵是一个二维数组A.edge[n][n]

$$\mathbf{A}.edge[i][j]=\begin{cases}\mathbf{1}, & \text{如果} <i,j>\in E \text{ 或者 } (i,j)\in E\\ \mathbf{0}, & \text{否则}\end{cases}$$

- 在有向图中，统计第i行1的个数可得顶点i的出度，统计第j列1的个数可得顶点j的入度
- 在无向图中，统计第i行(列)1的个数可得顶点i的度

邻接矩阵示例

- 无向图的邻接矩阵是对称的



$$\mathbf{A.edge} = \begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

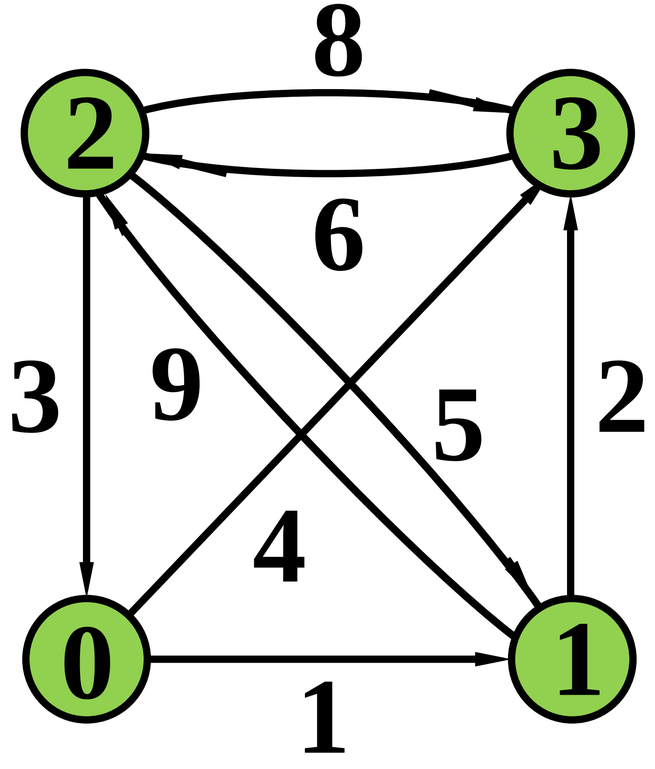
- 有向图的邻接矩阵可能是不对称的



$$\mathbf{A.edge} = \begin{array}{c} \\ 0 \\ 1 \\ 2 \end{array} \begin{array}{c} \begin{array}{ccc} 0 & 1 & 2 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix} \end{array}$$

网络的邻接矩阵

$$\mathbf{A.edge}[i][j]=\begin{cases}\mathbf{W}(i,j), & \text{若} i\neq j\text{且}<i,j>\in \mathbf{E}\text{或}(i,j)\in \mathbf{E}\\ \infty, & \text{若} i\neq j\text{且}<i,j>\notin \mathbf{E}\text{或}(i,j)\notin \mathbf{E}\\ \mathbf{0}, & \text{若} i==j\end{cases}$$



$$\mathbf{A.edge}=\begin{bmatrix}0 & 1 & \infty & 4\\ \infty & 0 & 9 & 2\\ 3 & 5 & 0 & 8\\ \infty & \infty & 6 & 0\end{bmatrix}$$

用邻接矩阵表示的图的类定义

```
const int EdgesMaxNum=100;  // 最大边数
const int VerticesMaxNum=20;  // 最大顶点数
template <class NameType, class DistType> class Graph {  // 图的类定义
  private:
    SeqList <NameType> Vertices(VerticesMaxNum);  // 顶点表
    DistType Edges[VerticesMaxNum][VerticesMaxNum];  // 邻接矩阵
    int EdgesCurrentNum;  // 当前边数
public:
    Graph(int size=VerticesMaxNum);  // 构造函数
    int GraphEmpty( ) const { return Vertices.IsEmpty( ); } // 判图空否
    int VerticesNumber( ) { return Vertices.Length(); }  // 返回当前顶点数
    int EdgesNumber( ) { return EdgesCurrentNumber; }  // 返回当前边数
```

邻接矩阵表示的图的类定义

```
    // 给出边(v1, v2)上的权
    DistType GetWeight(int v1, int v2);
    // 给出顶点v的第一个邻接顶点
    int GetFirstNeighborVer(int v);
    // 给出顶点v1的某邻接顶点v2后的下一个邻接顶点
    int GetNextNeighborVer(int v1, int v2);
    // 插入新顶点vertex
    void InsertVertex(const NameType &vertex);
    // 插入权为weight的新边(v1, v2)
    void InsertEdge(int v1, int v2, DistType weight);
    // 删除顶点vertex和所有与它相关联的边
    void RemoveVertex(int vertex);
    // 删除边(v1, v2)
    void RemoveEdge(int v1, int v2);
};
```

部分方法实现

```
template <class NameType, class DistType>
Graph <NameType, DistType> :: Graph(int size) {
    for (int i = 0; i < size; i++)  // 邻接矩阵初始化
        for (int j = 0; j < size; j++)
            Edges[i][j] = 0;
    EdgesCurrentNum = 0;  // 图中当前边数初始化
}

template <class NameType, class DistType>
DistType Graph <NameType, DistType> :: GetWeight(int v1, int v2) {
    if (v1 != -1 && v2 != -1)
        return Edges[v1][v2];
    else return 0;  // 带权图中权为0，表示无权值
}
```

部分方法实现

```
template <class NameType, class DistType>
int Graph <NameType, DistType> :: GetFirstNeighborVer(int v) {
    if (v != -1) {
        for (int col = 0; col <= Vertices.Length(); col++)
            if (Edges[v][col] > 0 && Edges[v][col] < MaxWeight)
                return col;
    }
    return -1;
}
```

部分方法实现

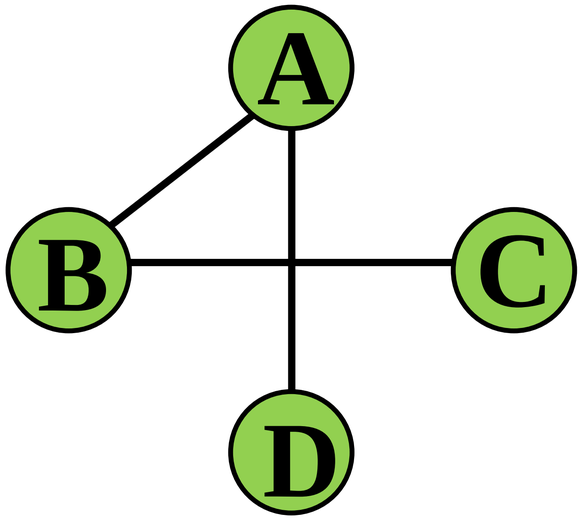
```
template <class NameType, class DistType>
int Graph <NameType, DistType> :: GetNextNeighborVer(int v1, int v2) {
    if (v1 != -1 && v2 != -1) {
        for(int col = v2 + 1; col <= Vertices.Length(); col++)
            if (Edges[v1][col] > 0 && Edges[v1][col] < MaxWeight)
                return col;
    }
    return -1;
}
```

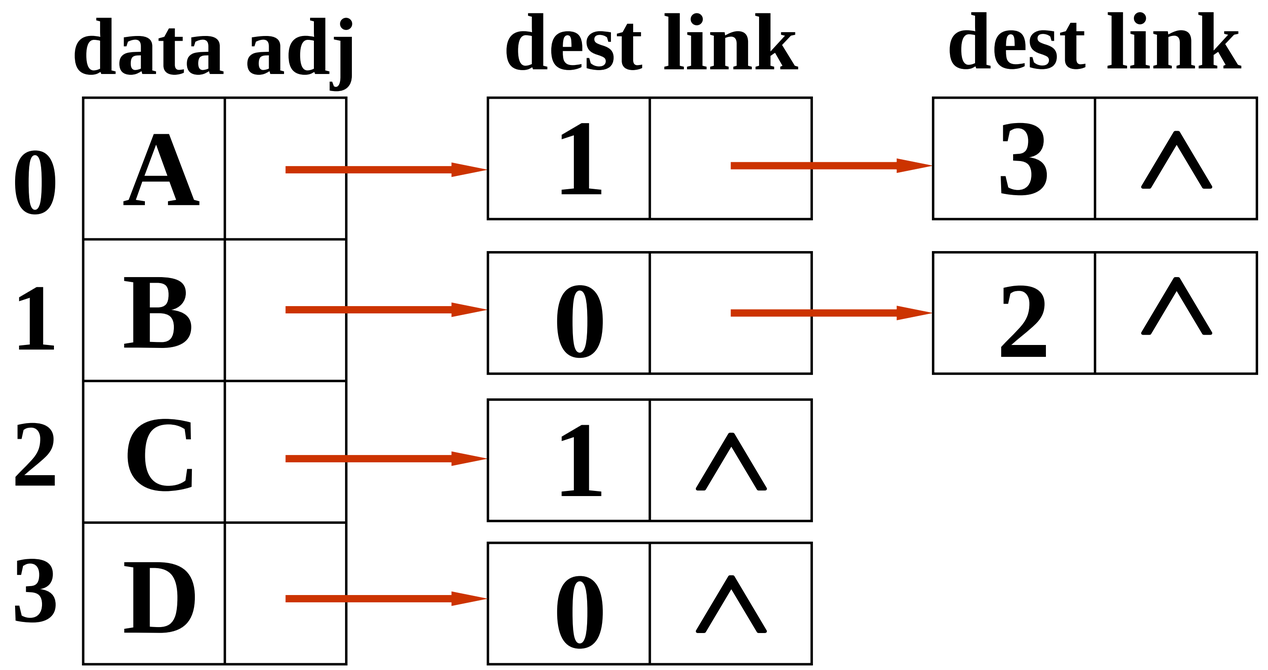
图的邻接表(Adjacent List)表示

- 当图的边数|E|远小于$|V|^2$的时候，邻接矩阵中会有大量的0或者∞元素，造成空间浪费，这时采用邻接表更合适

- 顶点i的边链表的表头指针adj在顶点表的下标为i的顶点记录中，该记录还保存了该顶点的其它信息data
- 同一个顶点发出的边链接在同一个边链表中，每一个链表结点代表一条边，链表结点中有另一顶点的下标dest和指向下一条边的指针link

无向图的邻接表示例

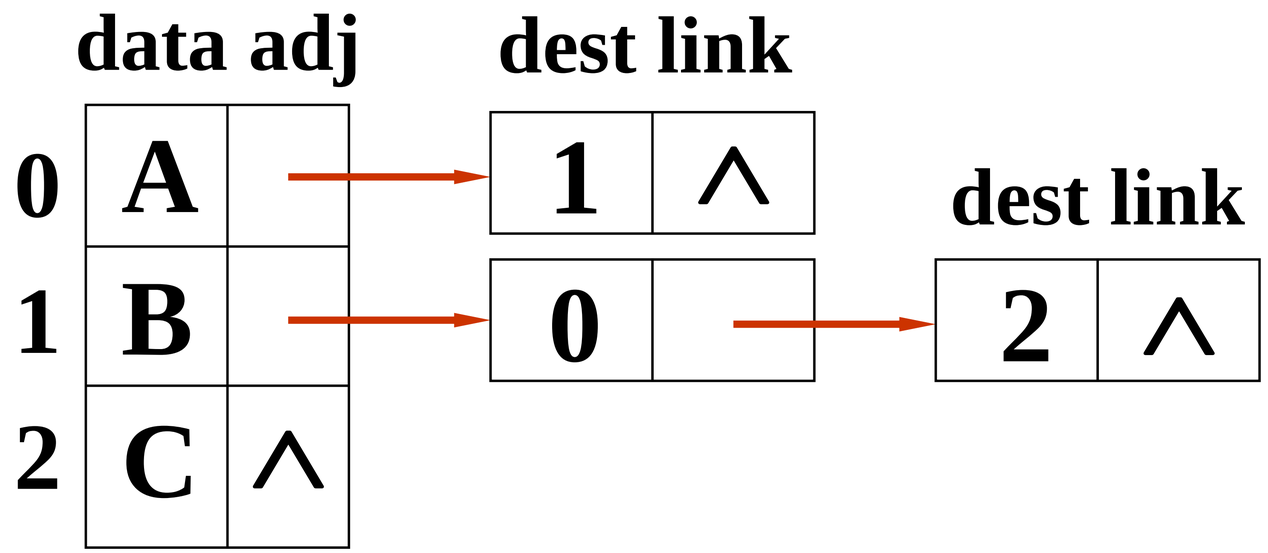




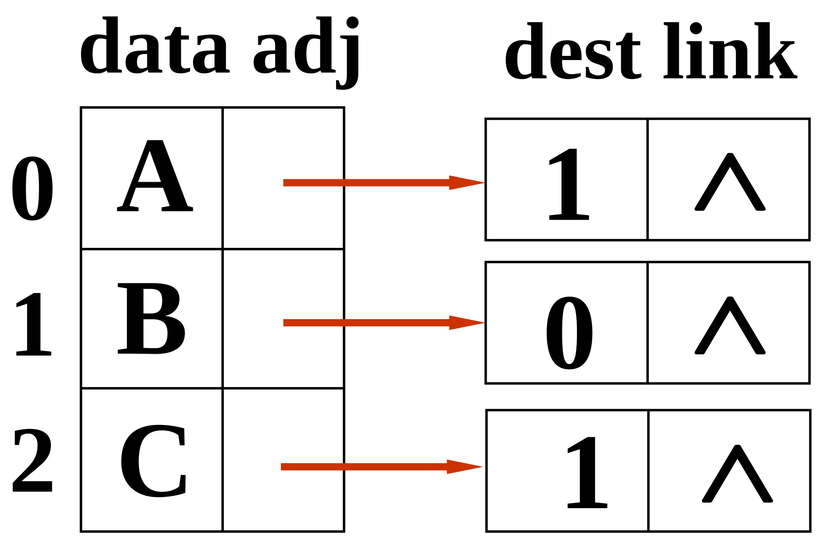
顶点表　　邻接表

有向图的邻接表示例





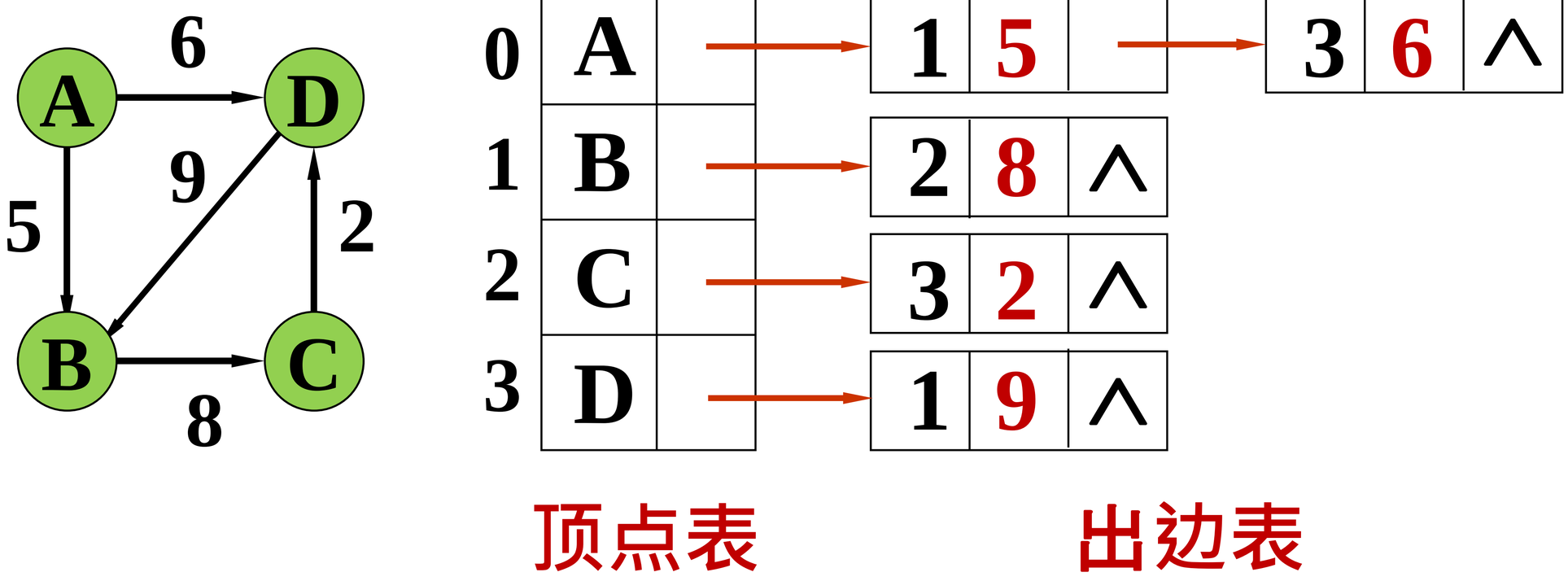
邻接表
(出边表)



逆邻接表
(入边表)

网络(带权图)的邻接表示例

- 带权图的边结点中保存该边上的权值cost



邻接表的特点

- 在邻接表的边链表中，各个边结点的链入顺序任意，视边结点输入次序而定

- 用邻接表表示**无向图**时，需要存储|V|个顶点结点和2|E|个边结点
- 用邻接表表示**有向图**时，若不考虑逆邻接表，只需要存储|V|个顶点结点和|E|个边结点

邻接表的边结点的类定义

```
const int DefaultSize =10;  // 缺省顶点个数
template <class NameType, class DistType> class Graph;
template <class DistType> class EdgeNode {  // 边结点的类定义
    friend class Graph <NameType, DistType>;
    private:
        int dest;  // 边的另一顶点位置
        DistType cost;  // 边上的权值
        EdgeNode <DistType> *link;  // 边结点后继指针
    public:
        EdgeNode( ) { }  // 构造函数
        EdgeNode(int d, DistType c) : dest(d), cost(c), link(NULL) { }
};
```

邻接表的顶点结点的类定义

```
// 顶点结点的类定义
template <class NameType, class DistType> class VertexNode {
    friend class EdgeNode <DistType>;
    friend class Graph <NameType, DistType>;
    private:
        NameType data;  // 顶点名字
        EdgeNode <DistType> *adj;  // 出边表的表头指针
    public:
        …
};
```

邻接表表示的图的类定义

```
template <class NameType, class DistType> class Graph {
    private:
        VertexNode <NameType, DistType> *NodeTable;  // 顶点表
        int VerticesNum;  // 当前顶点数
        int VerticesMaxNum;  // 最大顶点数
        int EdgesNum;  // 当前边数
        int GetVertexPos(const NameType &vertex);  // 顶点vertex的位置
public:
        Graph(int size);  // 构造函数
        ~Graph( );  // 析构函数
        int GraphEmpty( ) const { return VerticesNum == 0; }  // 判图空否
        int VerticesNumber( ) { return VerticesNum; }  // 返回图的顶点数
        int EdgesNumber( ) { return EdgeNum; }  // 返回图的边数
```

邻接表表示的图的类定义

```
// 给出边(v1, v2)上的权
DistType GetWeight(int v1, int v2);
// 给出顶点v的第一个邻接顶点
int GetFirstNeighbor(int v);
// 给出顶点v1的某邻接顶点v2后的下一个邻接顶点
        int GetNextNeighbor(int v1, int v2);
        // 插入新顶点vertex
        void InsertVertex(const NameType vertex);
        // 插入权为weight的新边(v1, v2)
        void InserEdge(int v1, int v2, DistType weight);
        // 删除顶点vertex和所有与它相关联的边
        void RemoveVertex(int vertex);
        // 删除边(v1, v2)
        void RemoveEdge(int v1, int v2);
};
```

部分方法实现

```
template <class NameType, class DistType> DistType
Graph <NameType, DistType> :: GetWeight(int v1, int v2) {
    // 若该边不存在于图中则返回权值0
    if (v1 != -1 && v2 != -1) {
        EdgeNode <DistType> *p = NodeTable[v1].adj;  // 边链表头指针
        while (p != NULL)
            if (p->dest==v2)
                return p->cost;  // 找到对应边，返回权值
            else p = p->link;  // 否则找下一条边
    }
    return 0;
}
```

部分方法实现

```
template <class nameType, class DistType> int
Graph <NameType, DistType> :: GetFirstNeighbor(int v) {
    // 如果不存在则返回-1
    if (v != -1) {  // v存在
        EdgeNode <DistType> *p = NodeTable[v].adj;  // 边链表头指针
        if (p != NULL)
            // 若第一个邻接顶点存在，则返回该边的另一个顶点
            return p->dest;
    }
    return -1;  // 若不存在，则返回-1
}
```

部分方法实现

```
template <class NameType, class DistType> int
Graph <NameType, DistType> :: GetNextNeighbor(int v1, int v2) {
    // 若没有下一个邻接顶点则返回-1
    if (v1 != -1) {  // v1存在
        EdgeNode <DistType> *p = NodeType[v1].adj;  // 边链表头指针
        while (p != NULL) {  // 寻找某邻接顶点v2
            if (p->dest == v2 && p->link != NULL)
                return p->link->dest;
            else p = p->link;
        }
    }
    return -1;
}
```

08 图

- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

图的遍历

- 图的遍历(Graph Traversal)：从连通图中某一顶点出发，沿着边访遍图中所有顶点，且每个顶点仅被访问一次
  - 深度优先搜索(Depth First Search, DFS)
  - 广度优先搜索(Breadth First Search, BFS)

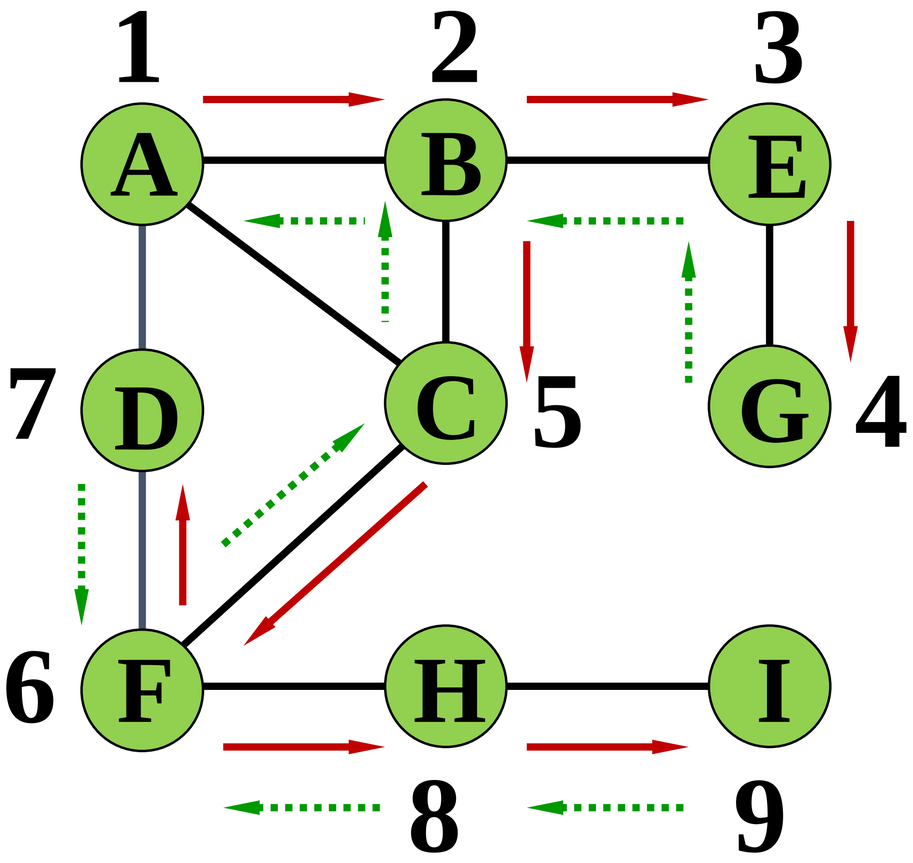
- 图中可能存在回路，且图的任一顶点都可能与其它顶点相连通，在访问完某个顶点之后可能会沿着某些边又回到了曾经访问过的顶点

- 为了避免重复访问，可设置一个标志顶点是否被访问过的辅助数组visited[ ]
  - visited[ ]的初始状态为0，表示未被访问过
  - 访问顶点i后，设visited[i]为1，表示已被访问过

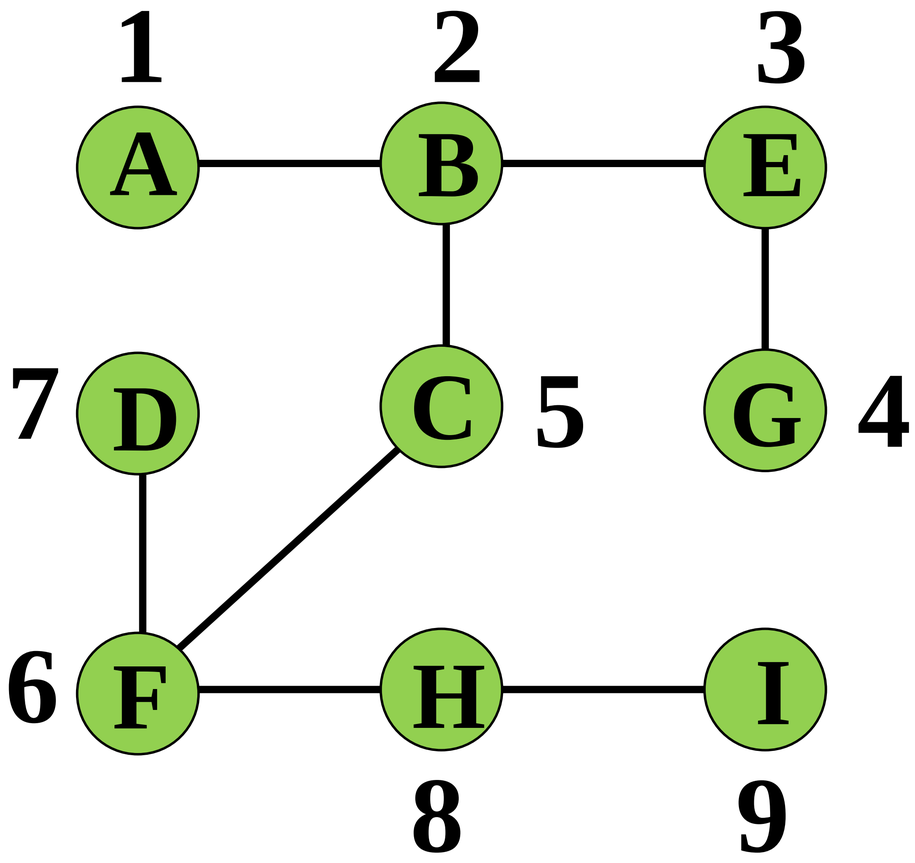
深度优先搜索

- 在访问图中某一起始顶点v后，由v出发，访问它的任一邻接顶点w_1，再从w_1出发，访问与w_1邻接但还没有访问过的顶点w_2，然后再从w_2出发，…如此进行下去，直至到达所有的邻接顶点都被访问过的顶点u为止
- 接着，退回一步，退到前一次刚访问过的顶点，看是否还有其它没有被访问的邻接顶点。如果有，则访问此顶点，之后再从此顶点出发，进行与前述类似的访问；如果没有，就再退回一步进行搜索。重复上述过程，直到连通图中所有顶点都被访问过为止

深度优先搜索示例



深度优先搜索过程



深度优先生成树

前进 ——→ 回退 ·····→

深度优先搜索算法

```
template <class NameType, class DistType> void
Graph <NameType, DistType> :: DFS( ) {
    int *visited = new int[VerticesNumber( )];  // 创建辅助数组
    for (int i = 0; I < VerticesNumber( ); i++)
        visited[i] = 0;  // 辅助数组初始化
    DFS(0, visited);  // 从顶点0开始进行深度优先搜索
    delete [ ] visited;
}
```

深度优先搜索算法

```
template <class NameType, class DistType> void
Graph <NameType, DistType> :: DFS(int v, int Visited[ ]) {
    cout<<v;  // 输出访问顶点信息
    visited[v] = 1;  // 该顶点访问标志修改为已访问过
    int w = GetFirstNeighbor(v);  // 寻找顶点v的第一个邻接顶点w
    while (w != -1) {  // 若邻接顶点存在
        if (!visited[w])  // 若该邻接顶点未被访问过
            DFS(w, visited);  // 则从邻接顶点w开始进行递归访问
        w = GetNextNeighbor(v, w);  // 寻找顶点v的下一个邻接顶点
    }
}
```

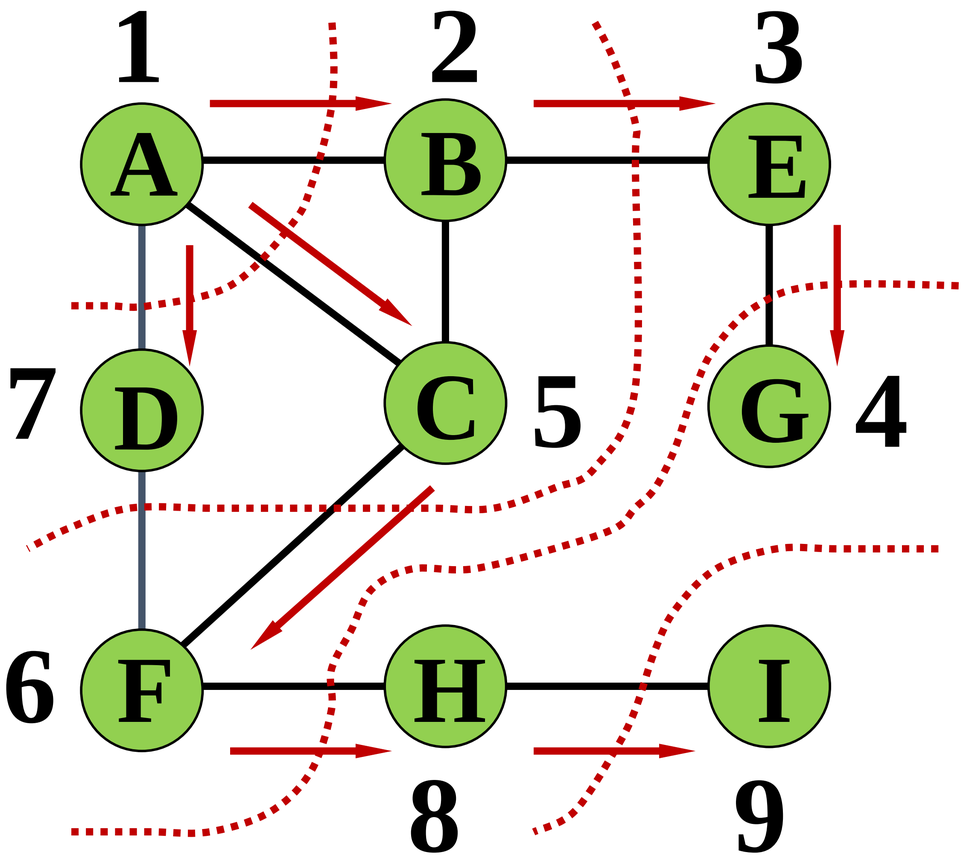
性能分析

- 邻接表表示，时间复杂度为$O(|V|+|E|)$
  - 沿邻接表可以依次取出顶点v的所有邻接顶点
  - 每个顶点只访问一次
- 邻接矩阵表示，时间复杂度为$O(|V|^2)$
  - 查找一个顶点的所有边需要$O(|V|)$
  - 需要访问|V|个顶点

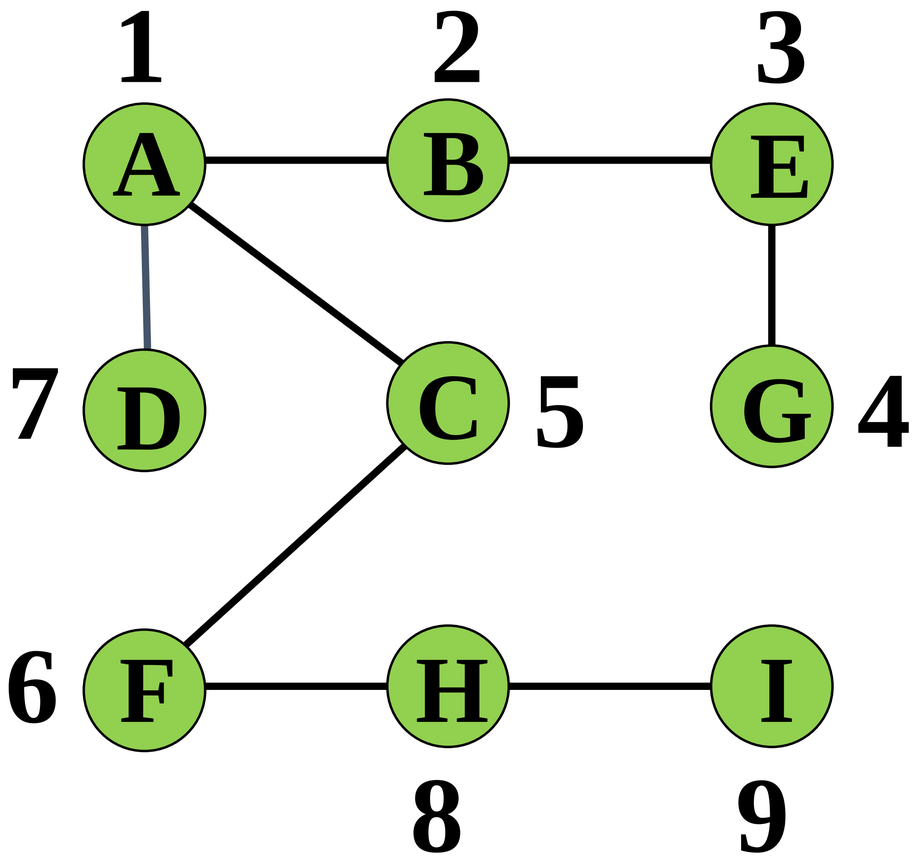
广度优先搜索

- 在访问了起始顶点v之后，由v出发，依次访问v的各个未被访问过的邻接顶点$w_1, w_2, \ldots, w_t$，然后再顺序访问$w_1, w_2, \ldots, w_t$的所有还未被访问过的邻接顶点
- 再从这些访问过的顶点出发，再访问它们的所有还未被访问过的邻接顶点，...如此下去，直到图中所有顶点都被访问为止
- 广度优先搜索是一种分层的搜索过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有往回退的情况。因此，广度优先搜索不是一个递归的过程
- 为了实现逐层访问，算法中使用了一个队列，以记忆正在访问的这一层和上一层的顶点，以便于向下一层访问

广度优先搜索示例



广度优先搜索过程



广度优先生成树

广度优先搜索算法

```
template <class NameType, class DistType> void
Graph <NameType, DistType> :: BFS(int v) {
    int *visited = new int [VerticesNumber( )];  // 创建辅助数组
    for (int i=0; i<VerticesNumber( ); i++) visited[i] = 0;  // 辅助数组初始化
    cout<<v;  visited[v] = 1;  // 该顶点访问标志修改为已访问过
    Queue <int> q;  q.EnQueue(v);  // 顶点v进队
    while (!q.IsEmpty( )) {
        v = q.Dequeue( );  // 从队列中退出顶点v
        int w = GetFirstNeighbor(v);  // 寻找顶点v的第一个邻接顶点w
        while (w != -1) {  // 邻接顶点w存在
            if (!visited[w]) { cout<<w<<‘ ’;  visited[w] = 1;  q.EnQueue(w); }
            w = GetNextNeighbor(v, w);  // 寻找顶点v的下一个邻接顶点
        }
    }
    delete [ ] visited;
}
```

性能分析

- 邻接表表示，时间复杂度为O(|V|+|E|)
  - 每个结点需要进队列一次
  - 每个结点的每条边要检查一次，共O(|E|)次
- 邻接矩阵表示， 时间复杂度为$O(|V|^2)$

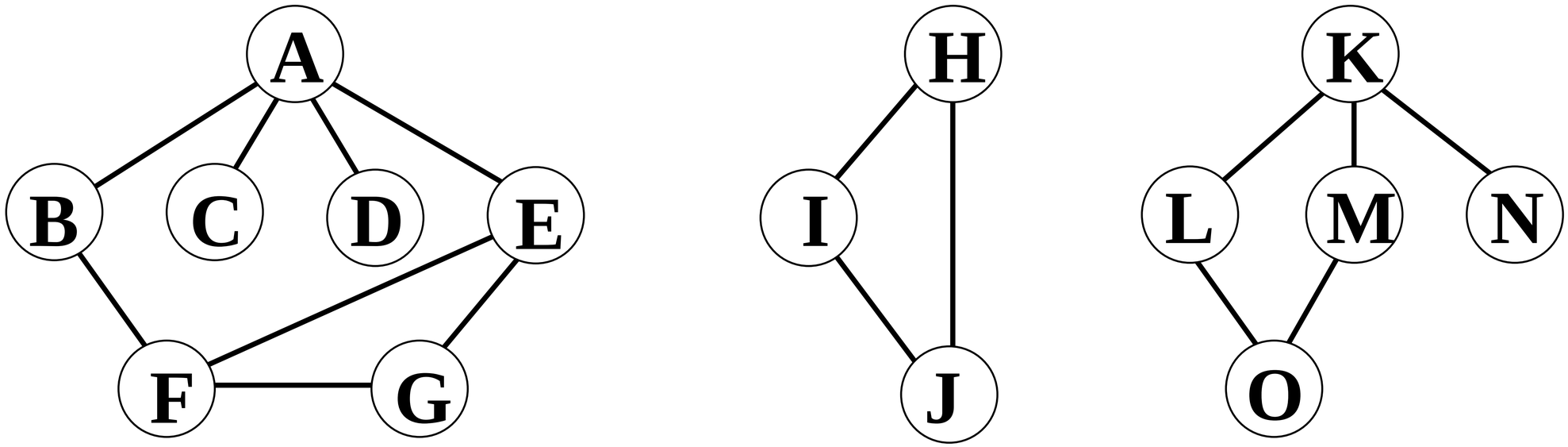
连通分量(Connected Component)

- 当无向图为非连通图时，从图中某一顶点出发，利用深度或广度优先搜索不可能遍历到图中的所有顶点，只能访问到该顶点所在的最大连通子图的所有顶点，而这样的最大连通子图就被称为连通分量

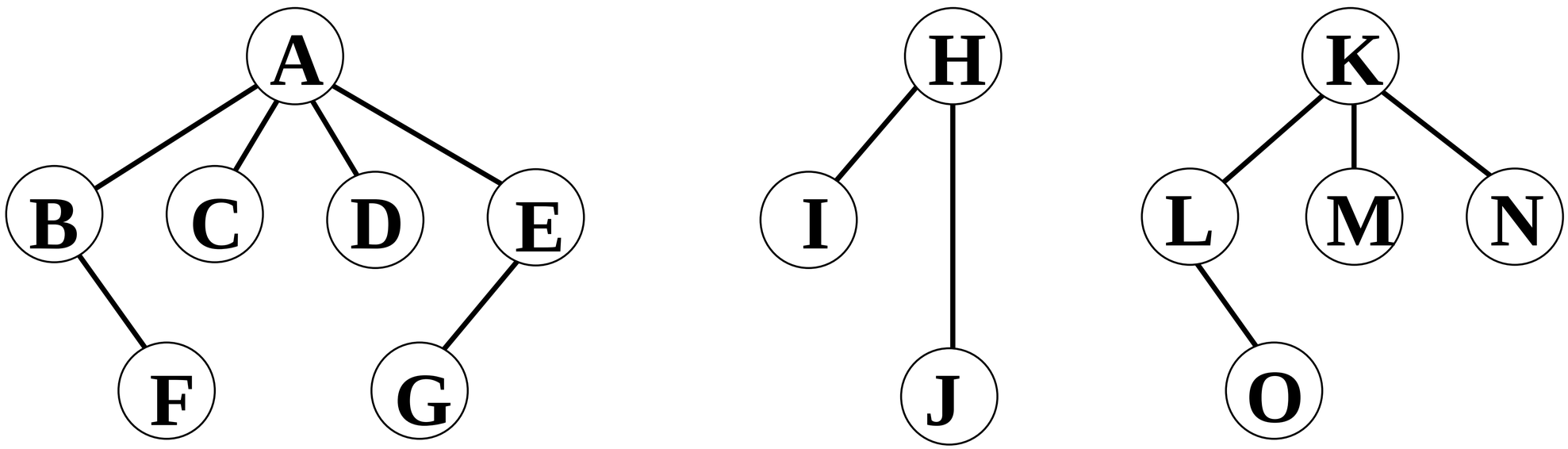
求连通分量

- 若从无向图的每一个连通分量中的一个顶点出发进行遍历，可求得无向图的所有连通分量
- 对图的每一个顶点进行检测：若已被访问过，则该顶点一定是落在图中已求得的连通分量上；若还未被访问，则从该顶点出发遍历图，可求得图的另一个连通分量
- 对于非连通的无向图，所有连通分量的生成树组成了非连通图的**生成森林**

求连通分量示例



由三个连通分量构成的非连通无向图



对应的生成森林

求连通分量算法

```
template <class NameType, class DistType> void
Graph <NameType, DistType> :: Components( ) {
    int n = G.VerticesNumber( );
    int *visited = new int[n];
    for (int i = 0; i < n; i++)  visited[i] = 0;
    for (i = 0; i < n; i++)
        if (!visited[i]) {  // 检测顶点是否访问过
            DFS(i, visited);  // 未访问，出发访问
            OutputNewComponent( );  // 输出连通分量
        }
    delete [ ] visited;
}
```

08 图

- 图的基本概念
- 图的存储结构
- 图的遍历
- **最小生成树**

最小生成树

- 设G是一个连通图，若G’是包含G中所有顶点的一个**无回路**的连通子图，则称G’为G的一棵**生成树**(spanning tree)， 若G有n个顶点，则G’有n个顶点和n-1条边
- 使用不同的遍历图的方法，可以得到不同的生成树；从不同的顶点出发，也可能得到不同的生成树
- 带权连通图G的所有生成树中边上权值之和最小的一棵称为G的**最小生成树**(minimum-cost spanning tree)

构造最小生成树的准则

- 必须使用且仅使用图中的n-1条边来连接图中的n个顶点
- 不能使用产生回路的边
- 各边上的权值总和达到最小
- 当各边有相同权值时，可以选择其中的任一条边，这时最小生成树可能不惟一

最小生成树的类定义

```
class MinSpanTree;
class MSTEdgeNode {  // 生成树边结点类
    friend class MinSpanTree;
    private:
        int tail, head;  // 生成树各边的两顶点
        float cost;  // 生成树各边的权值
    public:
        MSTEdgeNode( ) : tail (-1), head(-1), cost(0) { }
};
```

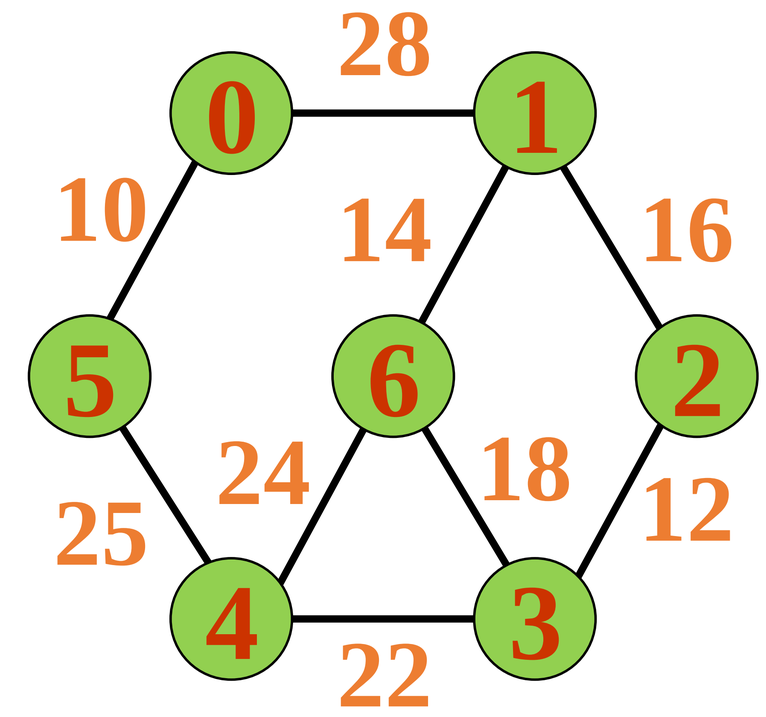
最小生成树的类定义

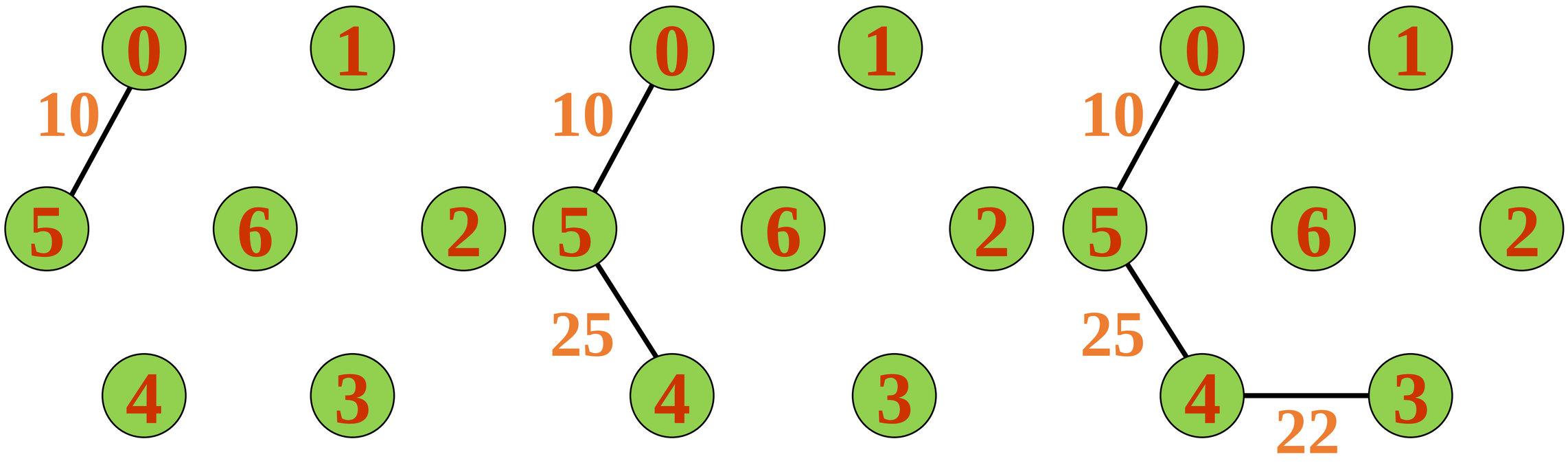
```
class MinSpanTree {
    protected:
        MSTEdgeNode *edgevalue;
        int MaxSize, n;
    public:
        MinSpanTree(int sz=VerticesNum-1) : MaxSize(sz), n(0) {
            edgevalue = new MSTEdgeNode[MaxSize];
        }
        int Insert(MSTEdgeNode &item);
};
```

普里姆(Prim)算法

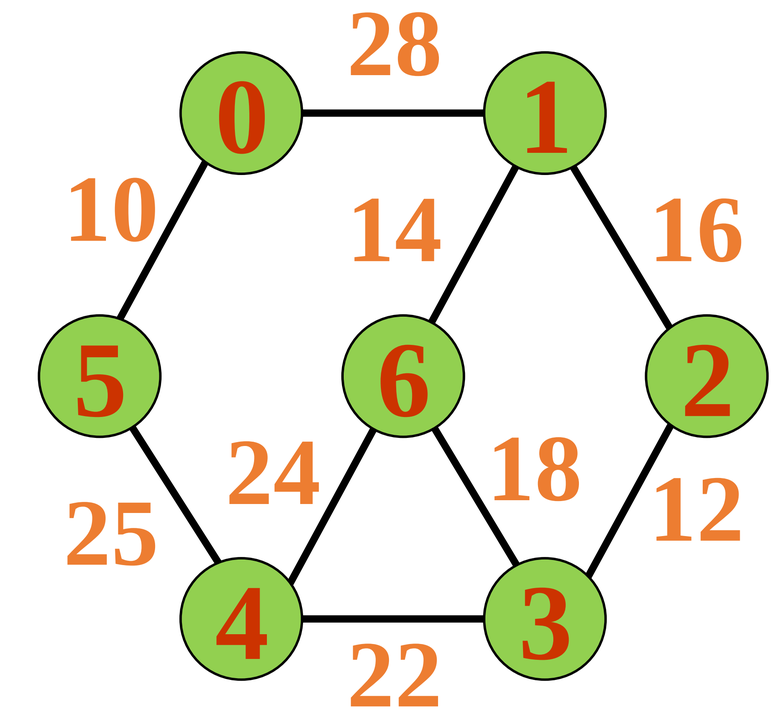
- 设G=(V, E)是一个带权连通图，顶点V={1, 2, ..., n}
- 设U为V中构成最小生成树的顶点集合，初始时U={v_0}, v_0是指定的某一个顶点且$v_0 \in V$
- 设T为构成最小生成树的边集合，初始时T为空。如果边(u, v)具有最小代价，且u∈U，v∈V-U，则最小生成树包含边(u, v)，即把v加到U中，把(u, v)加入T中。重复以上过程，直到U=V为止，这时T即为所求的最小生成树

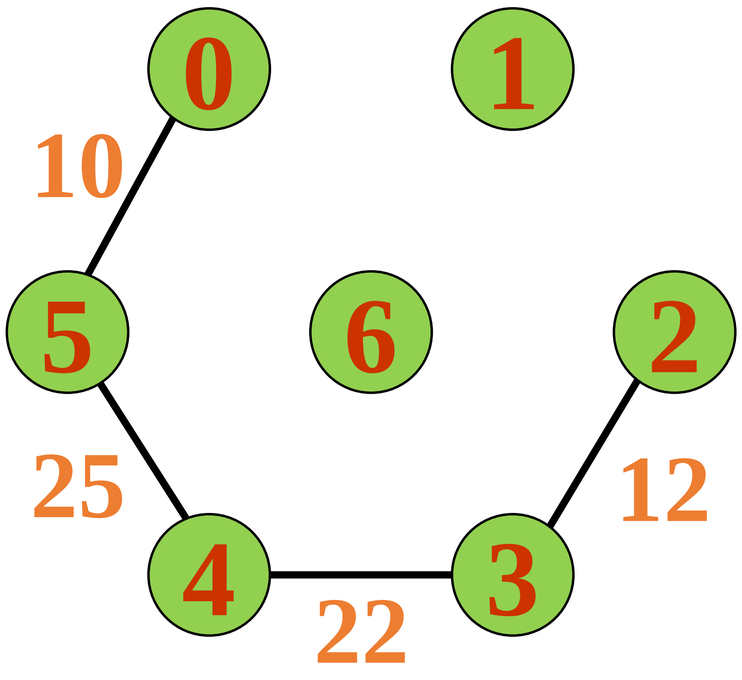
Prim算法过程示例

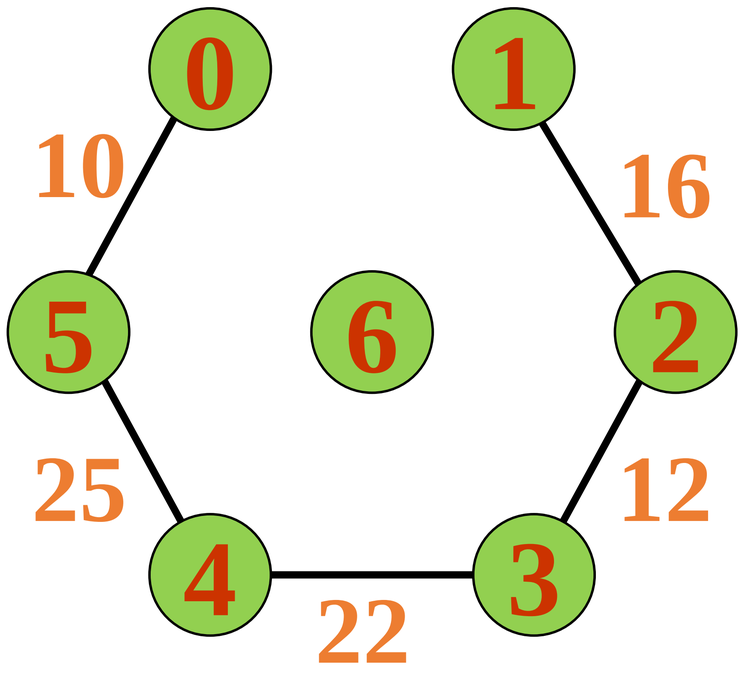

28
0
1
10
14
16
5
6
2
24
18
25
12
4
3
22

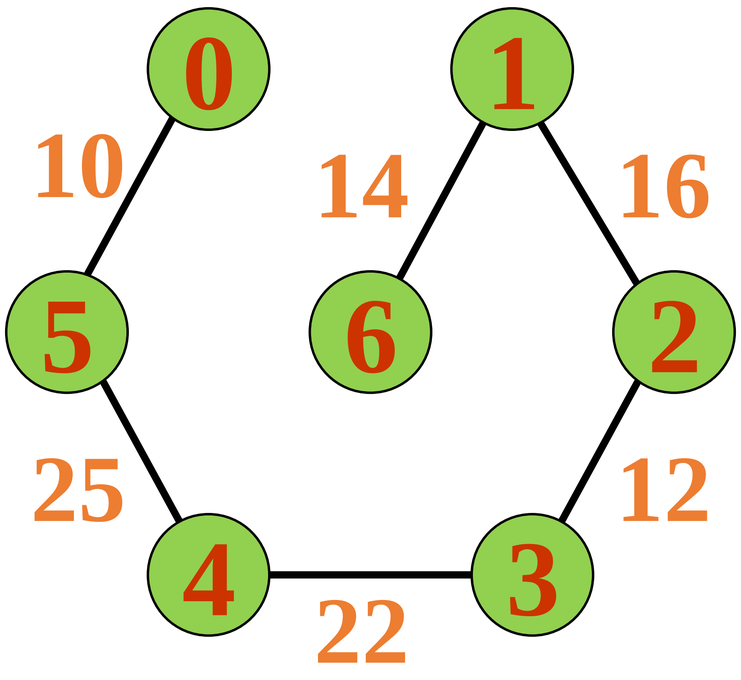

0
1
10
5
6
2
4
3
0
1
10
5
6
2
25
4
3
0
1
10
5
6
2
25
4
3
22

Prim算法过程示例


28
0
1
10
14
16
5
6
2
24
18
25
12
4
3
22


0
1
10
5
6
2
25
12
4
3
22


0
1
10
16
5
6
2
25
12
4
3
22

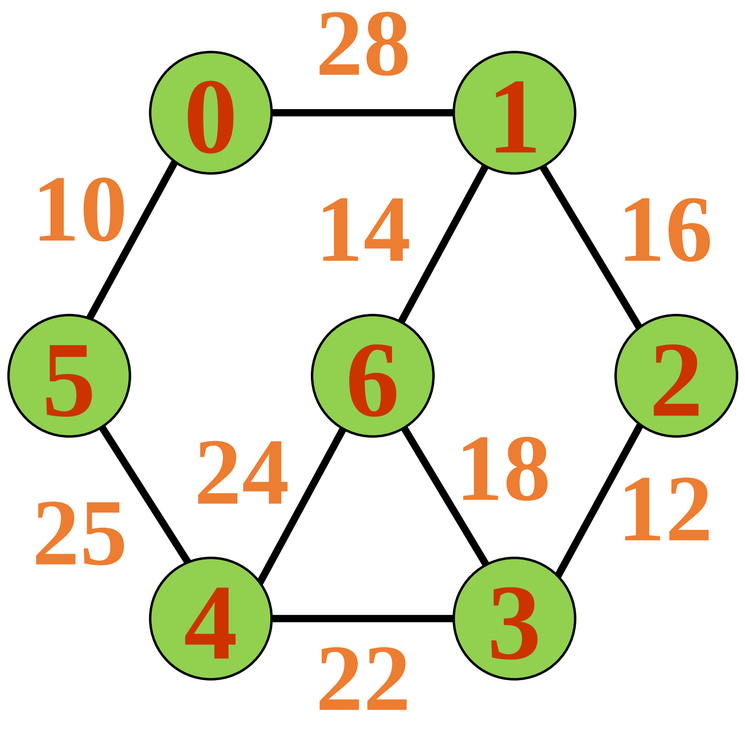

0
1
10
14
16
5
6
2
25
12
4
3
22

算法正确性证明

- 设T是带权连通图G=(V, E)的一棵正在建造的生成树，如果边(u, v)具有最小代价，且u∈U，v∈V-U，则G中包含T的最小生成树一定包含边(u, v)
  - 设T’是一棵包含T且不包含边(u, v)的最小生成树
  - 因为T’是树，所以它是连通的，故从u到v必有一条路径(u, ..., v)，路径(u, ..., v)中必有边(u’, v’)，满足u’∈U, v’∈V-U
  - 把(u, v)加入T’中，就构成一条回路
  - 由假设，边(u, v)的代价小于边(u’, v’)的代价(因为边(u, v)是连接U和V-U的最小代价的边)，在回路中删去(u’, v’)，从而破坏了这个回路，剩下的边构成另一棵生成树T”
  - T”包含边(u, v)，且各边的代价总和小于T’各边的代价总和，故T”是一棵包含边(u, v)的最小生成树。这样，T’不是G的最小生成树，这与假设相矛盾

采用的数据结构

- **在构造过程中，设置了两个辅助数组**
  - **nearvex[ ]：记录生成树顶点集合外各顶点距离集合内哪个顶点最近(即权值最小)** 关联数组
    - 已经加入生成树顶点集合的顶点的nearvex[ ]用-1作为标记
  - **lowcost[ ]：存放生成树顶点集合外各顶点到距离最近的已经加入生成树顶点集合的顶点的当前距离(即当前最小权值)**
    - 会在生成树扩大的过程中逐渐调整



$$\begin{pmatrix} 0 & 28 & \infty & \infty & \infty & 10 & \infty \\ 28 & 0 & 16 & \infty & \infty & \infty & 14 \\ \infty & 16 & 0 & 12 & \infty & \infty & \infty \\ \infty & \infty & 12 & 0 & 22 & \infty & 18 \\ \infty & \infty & \infty & 22 & 0 & 25 & 24 \\ 10 & \infty & \infty & \infty & 25 & 0 & \infty \\ \infty & 14 & \infty & 18 & 24 & \infty & 0 \end{pmatrix}$$

两个辅助数组的初始状态

- 若从顶点0出发，即u_0=0，则两个辅助数组的初始状态为：

	0	1	2	3	4	5	6
lowcost	0	28	∞	∞	∞	10	∞
nearvex	-1	0	0	0	0	0	0

- 然后，反复做以下工作：
  - 选择nearvex[i] != -1 && lowcost[i]最小的i，用v记住这个i，则选中的权值最小的边为(nearvex[v], v)，相应的权值为lowcost[v]

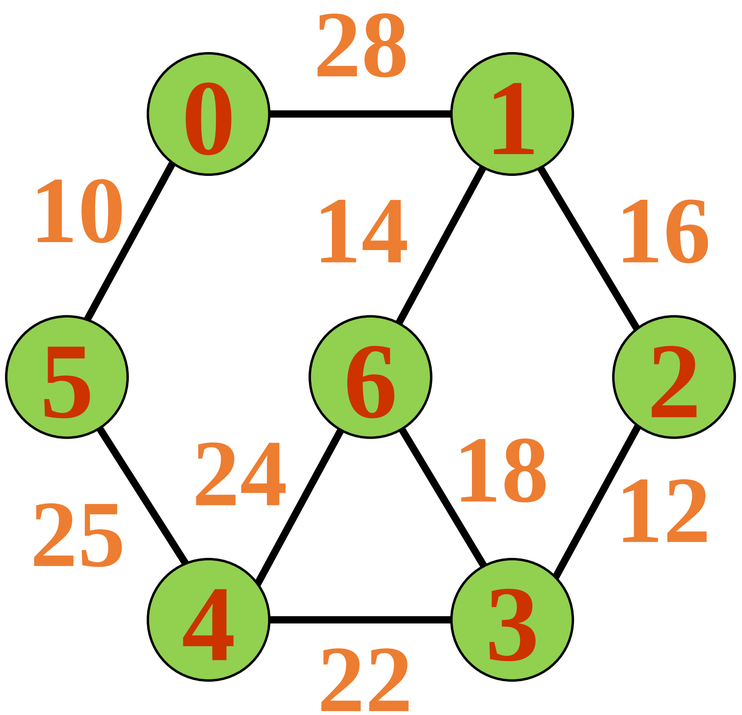
两个辅助数组的状态修正

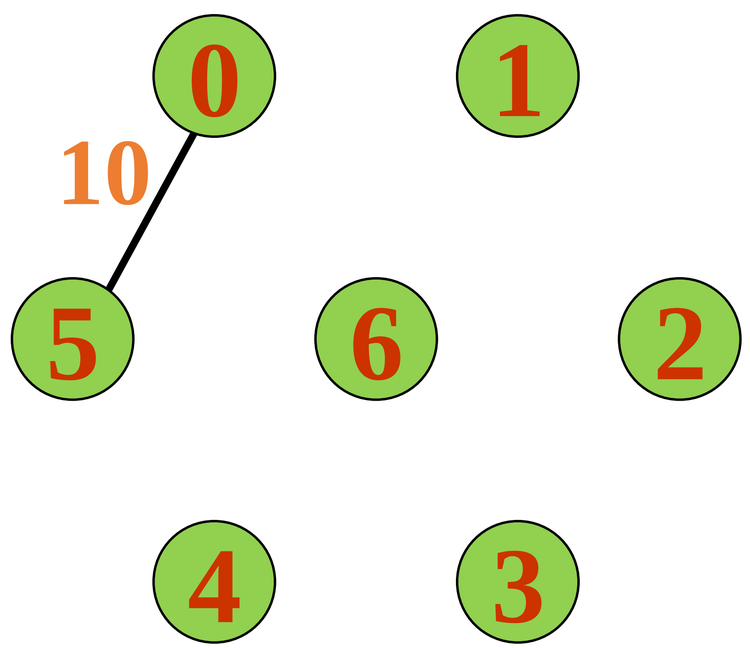
- 将边(nearvex[v], v, lowcost[v])加入生成树的边集合
- 将nearvex[v]改为-1，表示v已加入生成树顶点集合
- 用表达式lowcost[i] = min{lowcost[i], Edge[v][i]}修正生成树顶点集合外各顶点i对应的lowcost[i]
  - 即用顶点i到刚加入该集合的新顶点v的距离Edge[v][i]与原来到生成树顶点集合中顶点的最短距离lowcost[i]做比较，取距离近的作为新的lowcost[i]的值
  - 对修改了lowcost[i]的i，同时做修改nearvex[i] = v，表示生成树外顶点i到生成树内顶点v当前距离最近

首先选中了结点5

	0	1	2	3	4	5	6
lowcost	0	28	∞	∞	∞	10	∞
nearvex	-1	0	0	0	0	0	0

选v=5 ↑ 选边(0, 5)

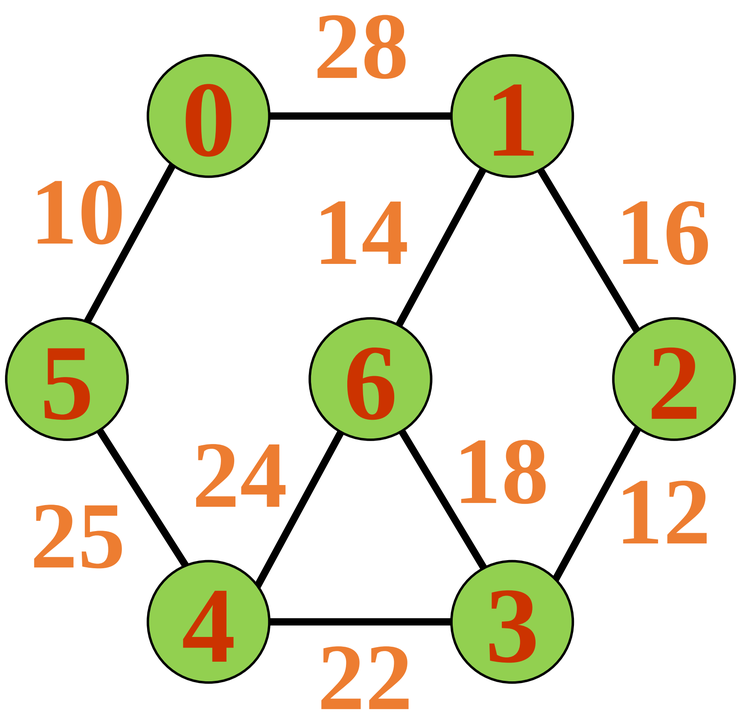


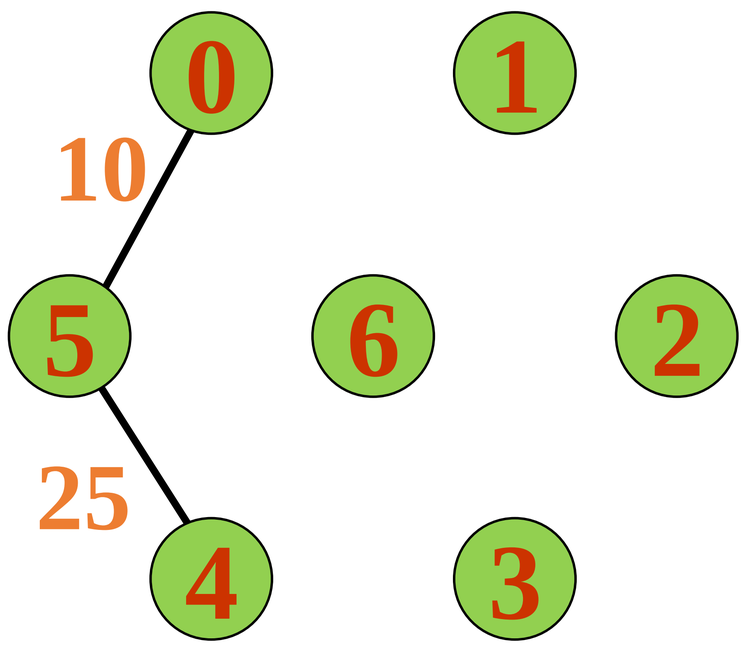


顶点v=5加入生成树顶点集合

	0	1	2	3	4	5	6
lowcost	0	28	∞	∞	25	10	∞
nearvex	-1	0	0	0	5	-1	0

选v=4　　选边(5, 4)

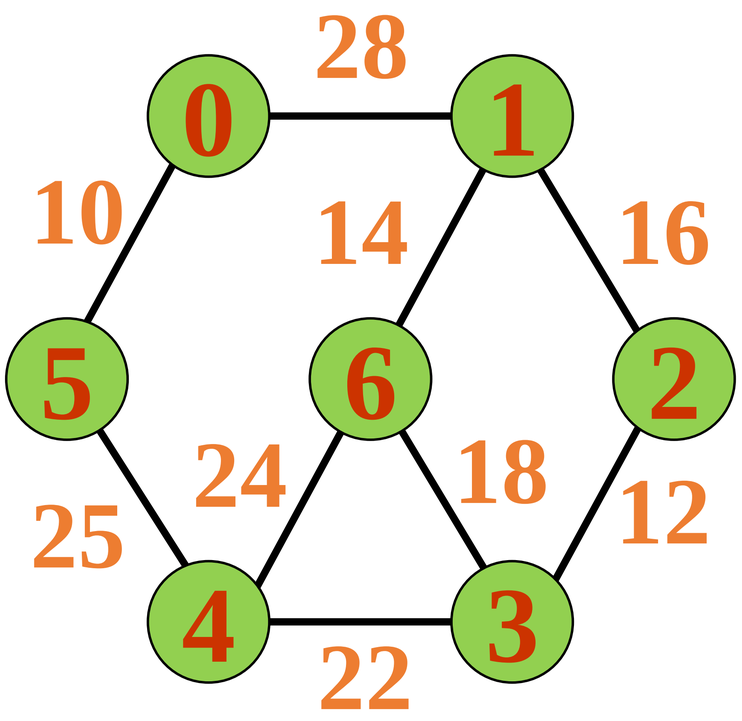


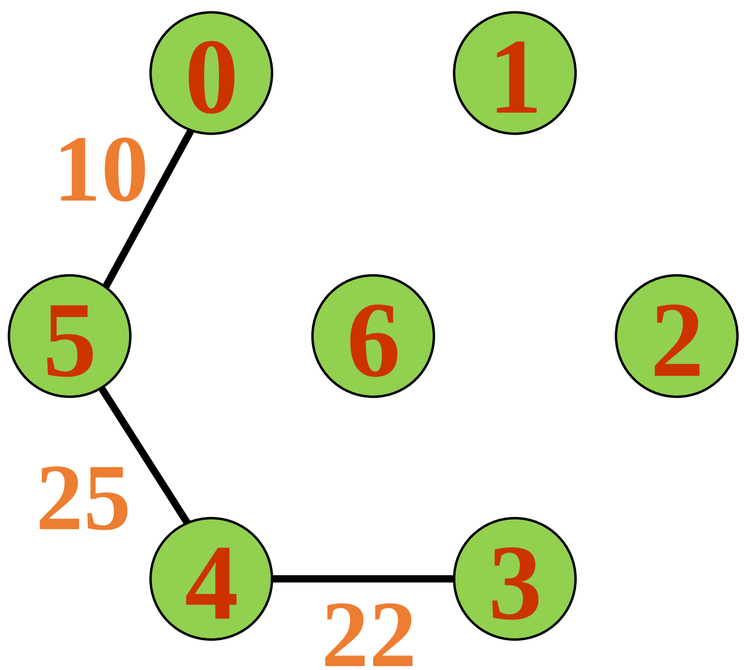


顶点v=4加入生成树顶点集合

	0	1	2	3	4	5	6
lowcost	0	28	∞	22	25	10	24
nearvex	-1	0	0	4	-1	-1	4

选v=3　　选边(4, 3)

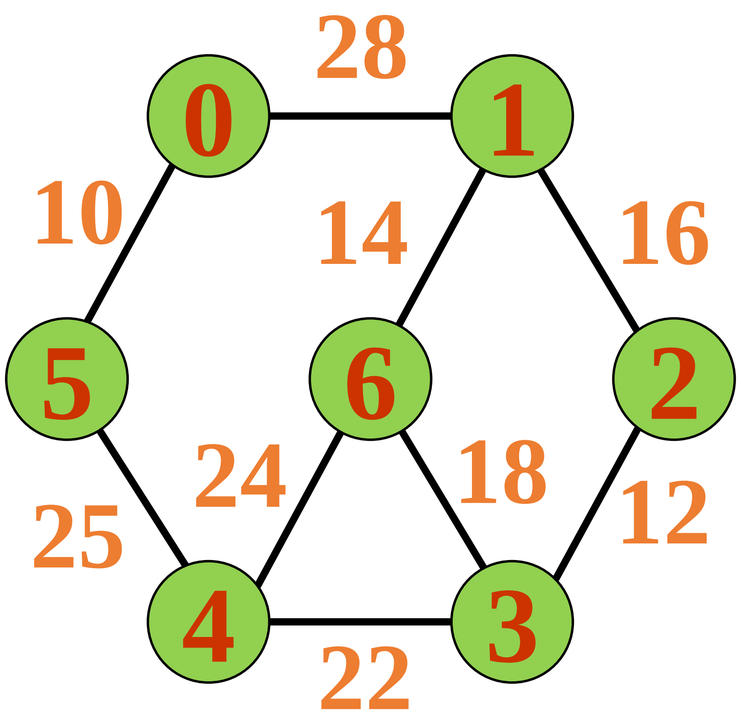


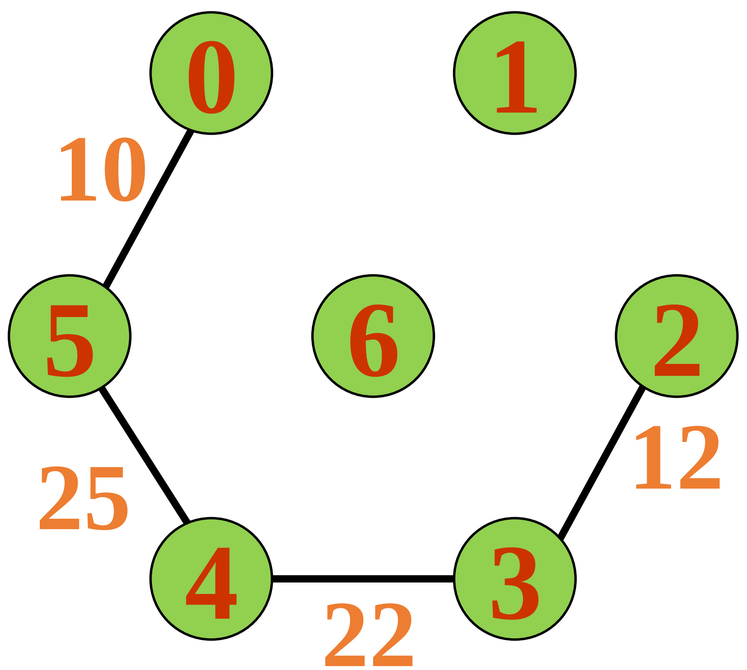


顶点v=3加入生成树顶点集合

	0	1	2	3	4	5	6
lowcost	0	28	12	22	25	10	18
nearvex	-1	0	3	-1	-1	-1	3

选v=2　　选边(3, 2)

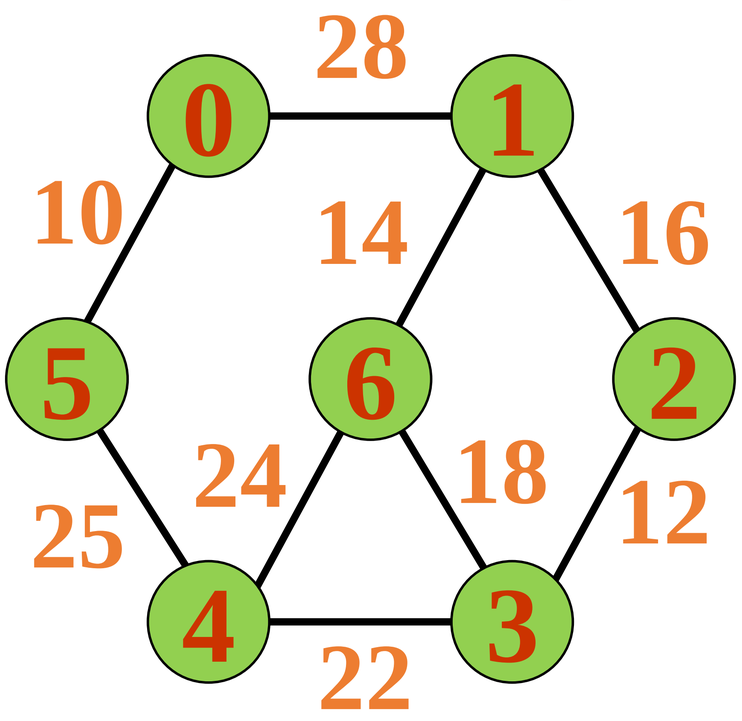


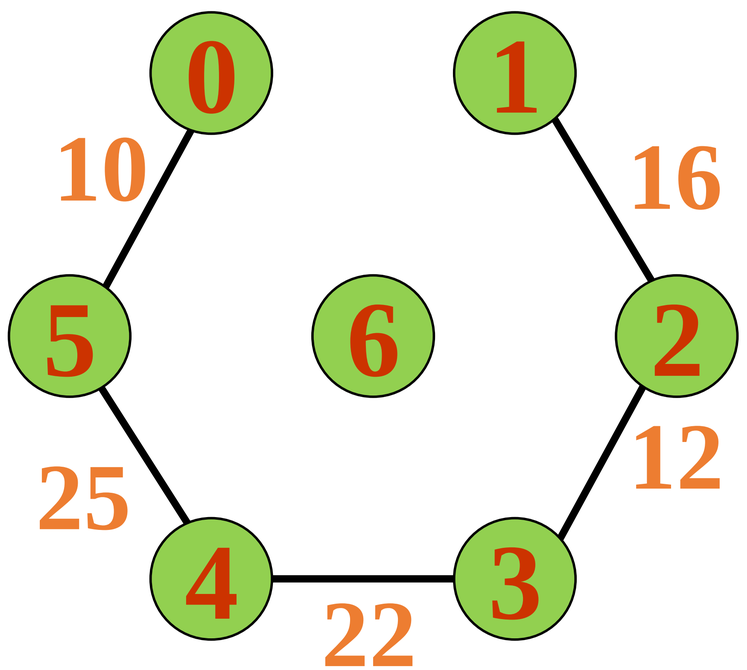


顶点v=2加入生成树顶点集合

	0	1	2	3	4	5	6
lowcost	0	16	12	22	25	10	18
nearvex	-1	2	-1	-1	-1	-1	3

选v=1　　选边(2, 1)

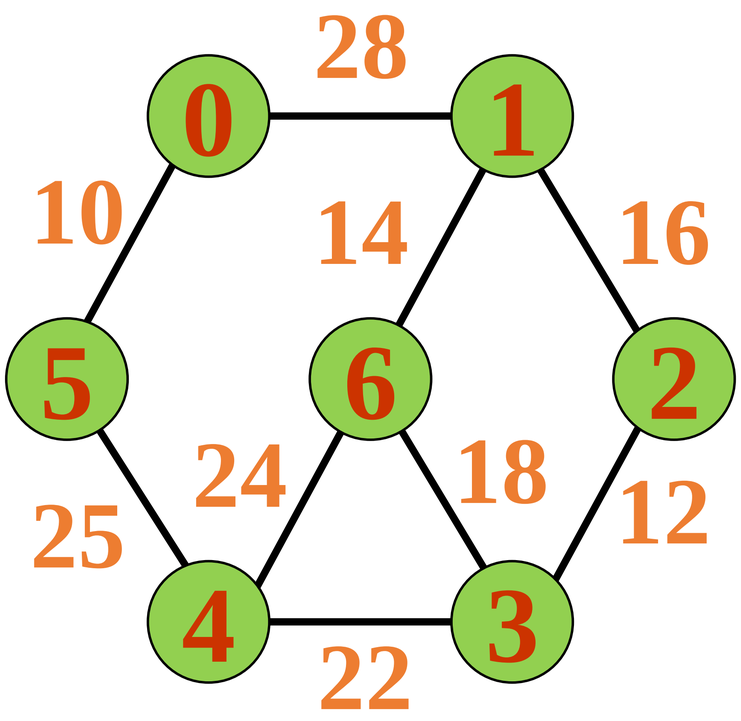


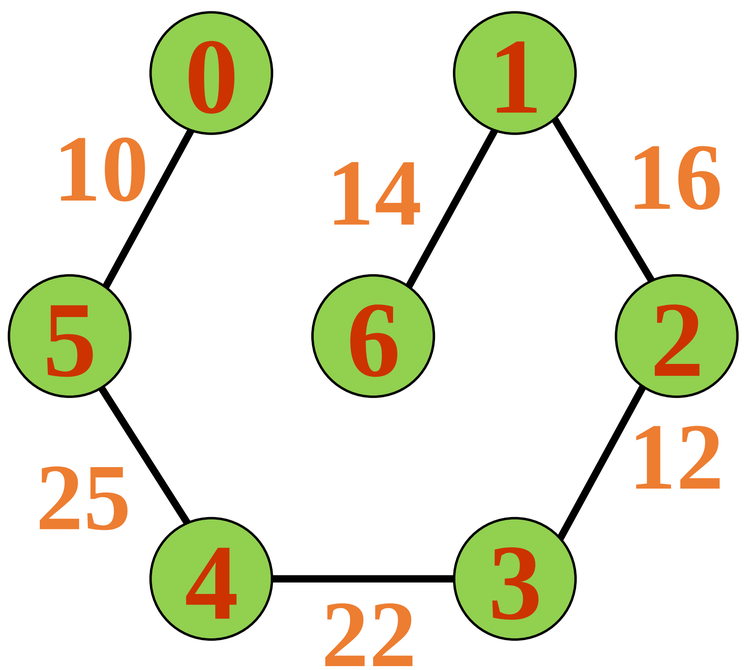


顶点v=1加入生成树顶点集合

	0	1	2	3	4	5	6
lowcost	0	16	12	22	25	10	14
nearvex	-1	-1	-1	-1	-1	-1	1

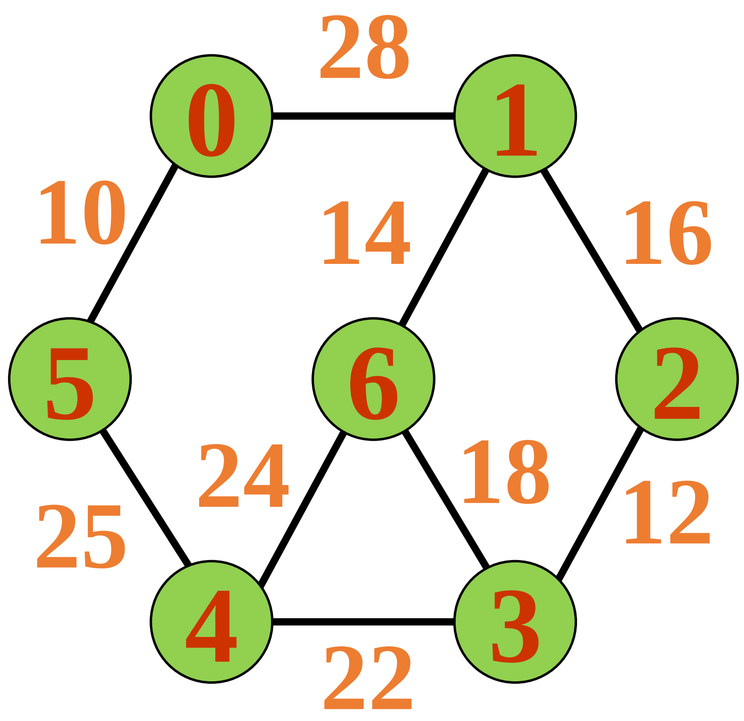
选v=6　　选边(1, 6)

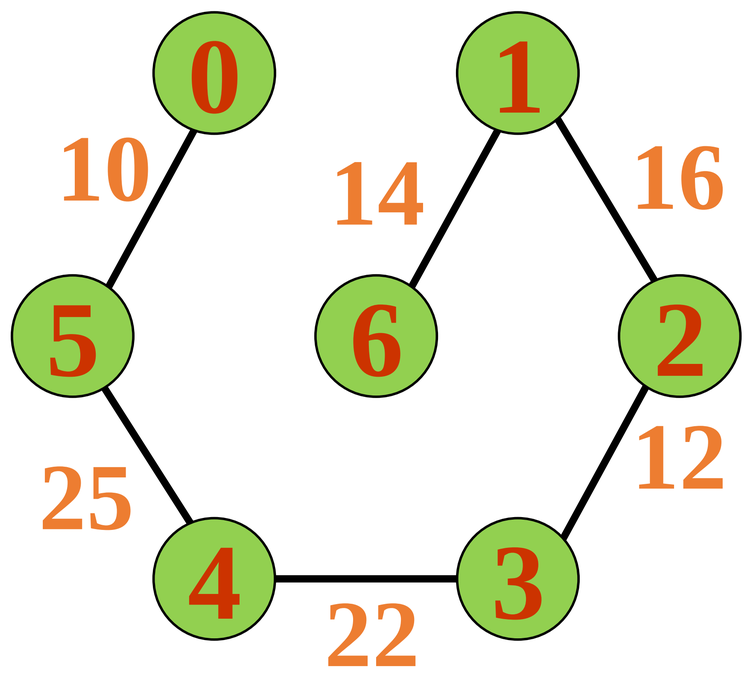




顶点v=6加入生成树顶点集合

	0	1	2	3	4	5	6
lowcost	0	16	12	22	25	10	14
nearvex	-1	-1	-1	-1	-1	-1	-1





Prim算法实现

```
void Prim(Graph <string, float> &G, MinSpanTree &T) {
    int i, j, n=G.VerticesNumber( ); MSTEdgeNode e;
    float *lowcost = new float[n]; int *nearvex = new int[n];
    for (i = 1; i < n; i++) { lowcost[i] = G.GetWeight(0, i); nearvex[i] = 0; }
    nearvex[0] = -1;  // 加到生成树顶点集合
    for (i = 1; i < n; i++) {  // 循环n-1次，加入n-1条边
        float min = MaxValue;  int v = 0;
        for (j = 1; j < n; j++)  // 求生成树外顶点到生成树内顶点最小权值的边
            if (nearvex[j] != -1 && lowcost[j] < min) { v = j; min = lowcost[j]; }
        if (v) {  // v=0表示再也找不到要求顶点        (tail, head)
            e.tail = nearvex[v]; e.head = v; e.cost = lowcost[v]; T.Insert(e); nearvex[v] = -1;
            for (j = 1; j < n; j++)  //更新
              if (nearvex[j] != -1 && G.GetWeight(v, j) < lowcost[j]) {
                lowcost[j] = G.GetWeight(v, j); nearvex[j] = v;
              }
        }
    }
}
```

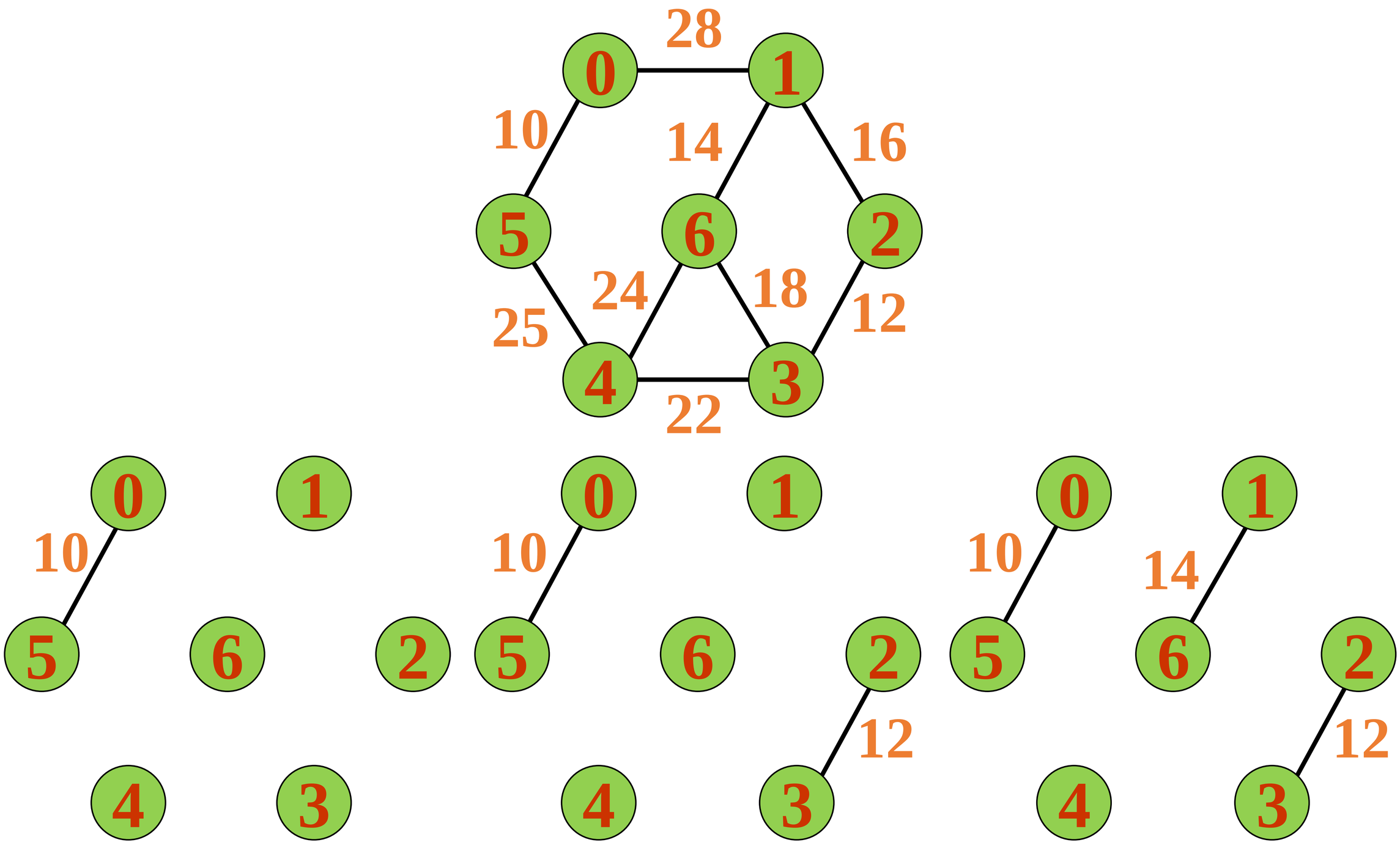
Prim算法性能分析

- 对于连通图G=(V, E)，该算法的时间复杂度为O(|V|²)
  - 与E无关，因此适用于边稠密的网络

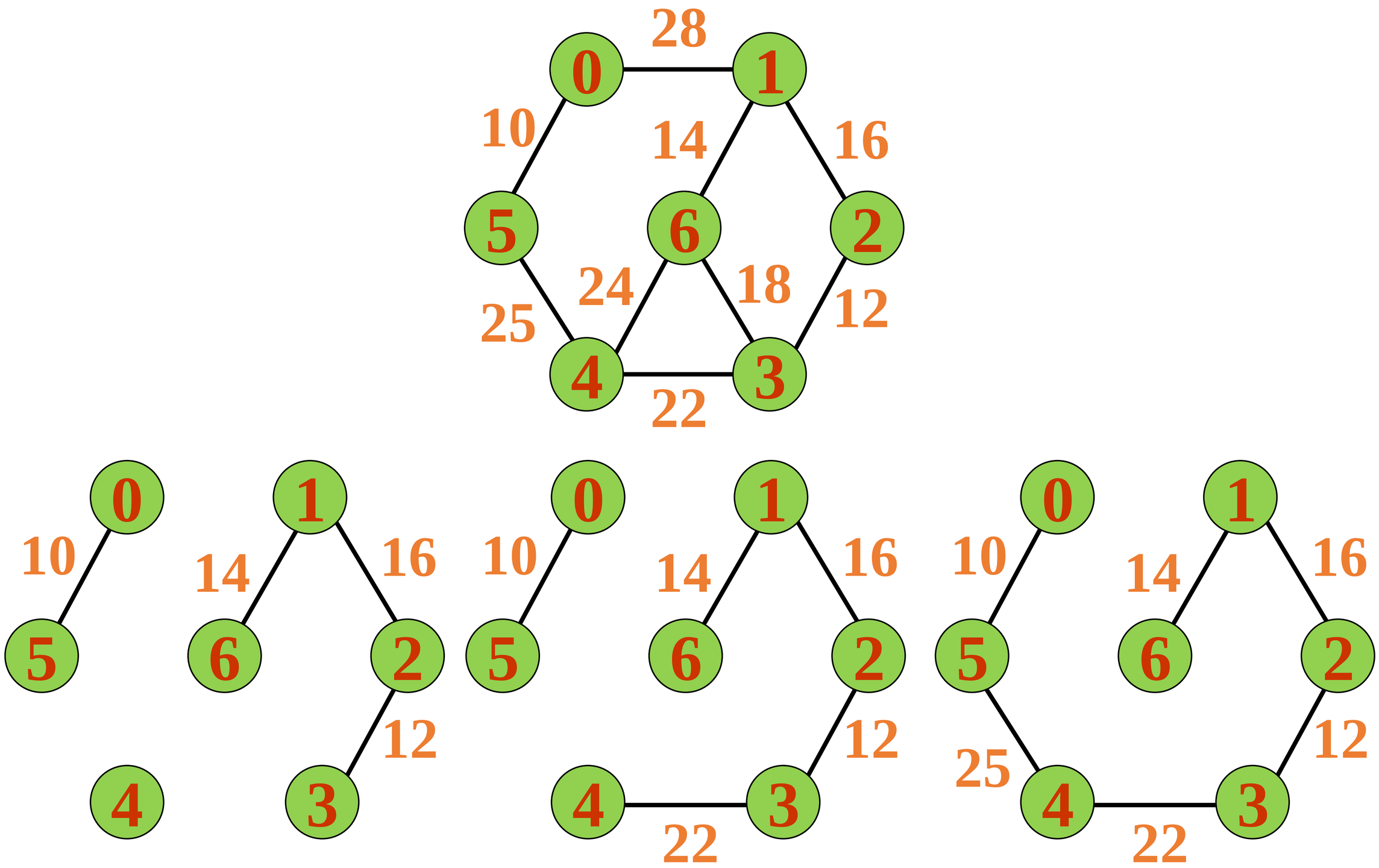
克鲁斯卡尔(Kruskal)算法

- 图G是一个具有n个顶点的带权连通图，设T是G的最小生成树，初始时T = (V, ∅)，即T由n个连通分量组成，每个连通分量只有一个顶点，没有边
- 首先，把E中的边按权值的递增次序进行排序，然后按排好序的顺序按以下步骤选取边。这样的过程一直进行到T包含有n-1条边为止，这时的T便是所求的最小生成树
- 选择步骤：若当前被选择的边的两个顶点在不同的连通分量中，则把这条边加到T中，选取这样的边可以保证不会构成回路；否则将此边舍去

Kruskal算法过程示例



Kruskal算法过程示例



Kruskal算法框架

- 主要数据结构是**最小堆**(Min-Heap)和**并查集**(Disjoint Set)
  - 最小堆存放E中所有的边，堆中每个结点的格式为(tail, head, cost)
  - 在构造最小生成树过程中，利用并查集检查依附一条边的两顶点tail和head是否在同一连通分量(即并查集的同一个子集合)上
    - 是则舍去这条边
    - 否则将此边加入T，同时将两个顶点合并到同一连通分量
- 重复这样的合并，直到所有顶点都合并到同一个连通分量
  - 这时T中所包含的边就构成了最小生成树

并查集

- **并查集(Disjoint Set，也叫Union-Find Set)是若干不相交的集合，能够快速判断元素所在集合、快速合并集合**
- **并查集支持以下三种操作：**
  - **Union(Root1, Root2) // Root2合并入Root1**
  - **Find(x) // 搜索元素x所在的集合**
  - **UFSets(s) // 将并查集中s个元素初始化为s个只有1个元素的子集合**
- **并查集的典型实现方法是每个集合用一棵树表示**
  - **采用树的双亲表示，集合元素放在一个数组parent中，编号从0到n-1，其中n是最大元素个数**
  - **数组下标i表示第i个元素对应的树结点；parent[i]的值是结点i的双亲结点在数组中的下标；根结点的双亲为负数，其绝对值表示集合中的元素个数** 包括自身
  - **在同一棵树上所有结点所代表的集合元素在同一个集合中**

并查集初始状态

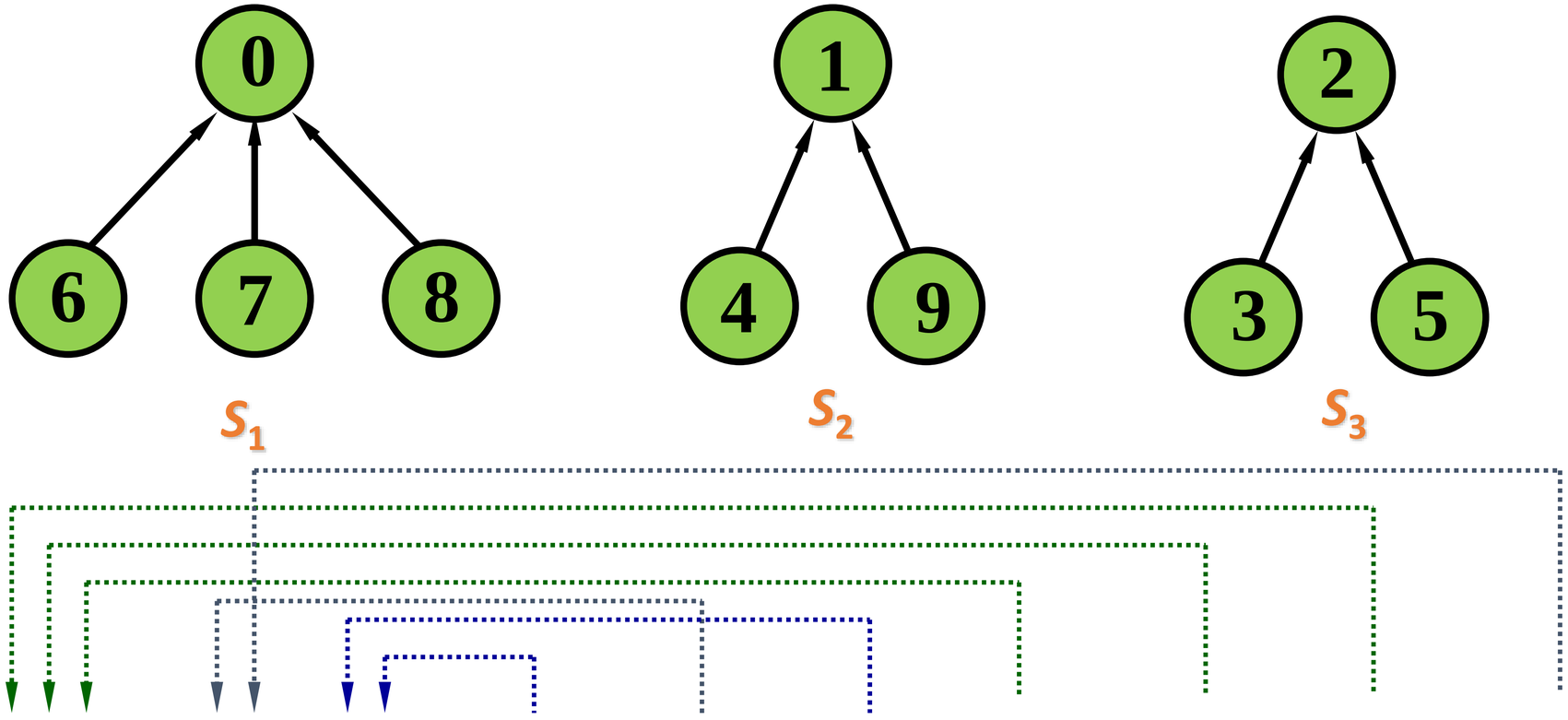
- 初始时，用构造函数UFSets(s)构造一个森林，每棵树只有一个结点，表示集合中各元素自成一个子集合

下标	0	1	2	3	4	5	6	7	8	9
parent	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

- 用Find(i)寻找集合元素i的根。如果有两个集合元素i和j，Find(i) == Find(j)，表明这两个元素在同一个集合中
- 如果两个集合元素i和j不在同一个集合中，可用Union(i, j)将它们合并到一个集合中

并查集示例

- 3个集合S1 = {0, 6, 7, 8}，　S2 = {1, 4, 9}，　S3 = {2, 3, 5}，对应的数组表示为

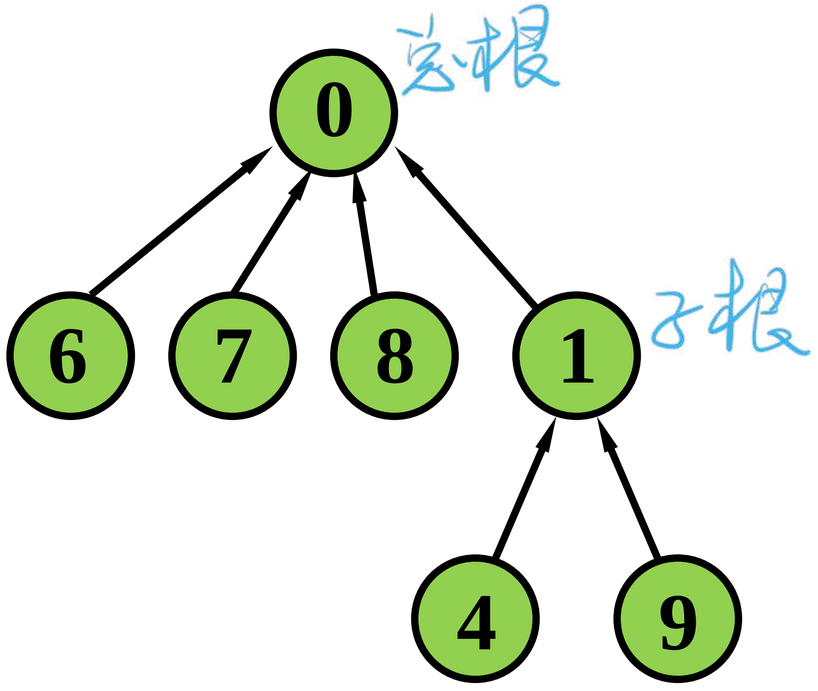


下标	0	1	2	3	4	5	6	7	8	9
parent	-4	-3	-3	2	1	2	0	0	0	1

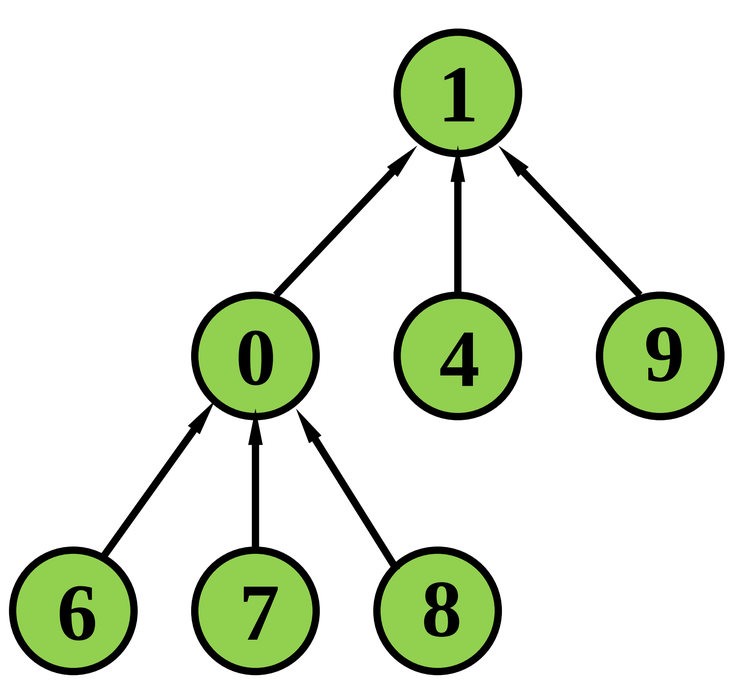
并查集合并示例

下标	0	1	2	3	4	5	6	7	8	9
parent	-7	0	-3	2	1	2	0	0	0	1

集合 $S_1 \cup S_2$ 和 S_3 的双亲表示



$S_1 \cup S_2$



S1 ∪ S2 还可以是这样

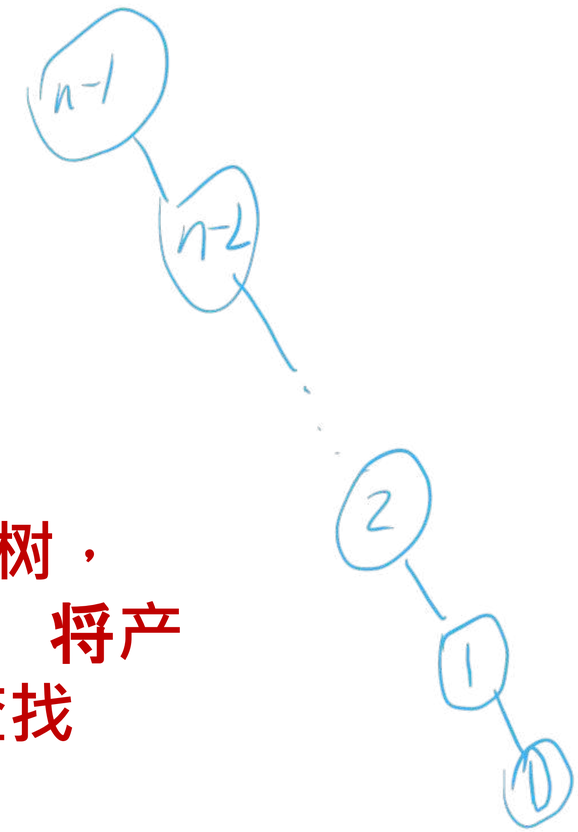
并查集的类定义

```
const int DefaultSize = 10;
class UFSets {  // 并查集类定义
    private:
        int *parent;  // 集合元素数组
        int size;  // 集合元素的数目
    public:
        UFSets (int s = DefaultSize) {  // 构造函数
            size = s;  parent = new int [size];
            for ( int i = 0; i < size; i++ ) parent[i] = -1;
        }
        ~UFSets ( ) { delete [ ] parent; }  // 析构函数
        const UFSets & operator = (UFSets &R);  // 重载函数：集合赋值
        void Union (int Root1, int Root2);  // 两个子集合合并
        int Find (int x);  // 搜寻集合x的根
        void WeightedUnion (int Root1, int Root2);  // 小集合向大集合合并
};
```

并查集操作的算法

```
int UFSets :: Find (int x) {
    while (parent[x] >= 0)
        x = parent[x]; //一直向上找，直到为根
    return x;
}

void UFSets :: Union (int Root1, int Root2) {
    // 将Root2连接到Root1下面
    parent[Root1] += parent[Root2];
    parent[Root2] = Root1;
}
```

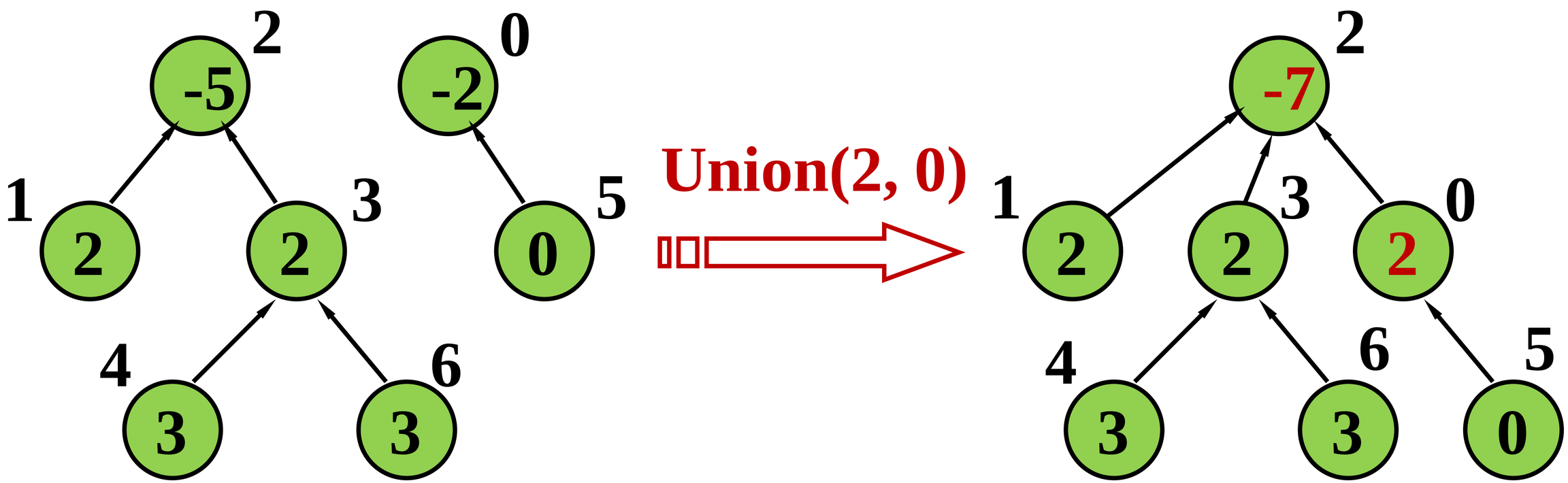


Find和Union操作性能不好。n棵由1个结点构成的树，做处理Union(n-2, n-1), …, Union(1, 2), Union(0, 1)， 将产生的都是单枝树，Find操作就变成了线性表上的查找

改进的方法

- 向结点个数多的树合并
- 向比较高的树合并(略)
- 压缩各结点的路径长度(略)

向结点个数多的树合并



向结点个数多的树合并

```
void UFSets :: WeightedUnion (int Root1, int Root2) {
    int temp = parent[Root1] + parent[Root2];
    if ( parent[Root2] < parent[Root1] ) {  //负数
        parent[Root1] = Root2;  // Root2 中结点数多
        parent[Root2] = temp;  // Root1 指向 Root2
    } else {
        parent[Root2] = Root1;  // Root1 中结点数多
        parent[Root1] = temp;  // Root2 指向 Root1
    }
}
```

Kruskal算法实现

```
void Kruskal(Graph <string, float> &G, MinSpanTree &T) {
    MSTEdgeNode ed;
    int n = G.VerticesNumber ( );  int e = G.EdgesNumber ( );
    MinHeap <MSTEdgeNode> H(e);  UFSets F(n);
    for (int u = 0; u < n; u++) //建边堆
        for (int v = u + 1; v < n; v++)
            if (G.GetWeight(u, v) != MaxValue) {
                ed.tail = u;  ed.head = v; ed.cost = GetWeight(u, v);  H.Insert(ed);
            }
    int count = 1;  // 最小生成树加入边数计数
    while (count < n) {
        H.RemoveMin(ed);  u = F.Find(ed.tail);  v = F.Find(ed.head);
        if (u != v) { F.Union(u, v);  T.Insert(ed);  count++; }
    }
}
```

Kruskal算法性能分析 – 采用邻接矩阵

- 在建最小堆的循环中，检测邻接矩阵需要$O(|V|^2)$，而|E|条边构成初始最小堆需要$O(|E|\log_2|E|)$
- 在构造最小生成树中，需要$O(|E|)$次出堆操作、2|E|次Find()和|V|-1次Union，计算时间分别为$O(|E|\log_2|E|)$、$O(|E|\log_2|V|)$和$O(|V|)$
- 故总时间为$O(|E|\log_2|E| + |E|\log_2|V| + |V|^2 + |V|)$
- 调整并查集，Find()操作只要常数时间，即$O(|E|\log_2|V|)$还可以降为$O(|E|)$，故总时间$O(|E|\log_2|E|+|V|^2)$

Kruskal算法性能分析 – 采用邻接表

- 修改建最小堆的程序，其检测邻接表各边结点的时间可达到O(|V|+|E|)，|E|条边构成初始最小堆需要O(|E|$\log_2$|E|)
- 构造最小生成树的时间代价与上一页的相同，仍然是O(|E|$\log_2$|E| + |E| + |V|)
- 故总时间为O(|E|$\log_2$|E| + |V|)
- 相对于Prim算法，更适合边稀疏的情况

本章小结

- 图的基本概念
- 图的存储结构
- 图的遍历
- 最小生成树

作业

- G是有n个顶点的连通图，证明G至少有n-1条边
- 改造深度优先搜索算法，在遍历图的过程中建立生成森林的左子女-右兄弟链表
- 给定一个连通图G，所有边都没有权值，编写一个算法求从顶点v能到达的最短路径为k的所有顶点(最短路径长度以边数为单位计算)